CHAPTER 1

INTRODUCTION

Artificial Intelligence is a branch of computer science that enables machines to perform tasks that typically require human intelligence, such as reasoning, pattern recognition, perception, and decision making. Artificial Intelligence systems use algorithms that can interpret complex data, learn from experience, and adapt to new information. In the field of biometric security, Artificial Intelligence has become indispensable for building systems that can reliably verify human identity while resisting attempts to deceive them using forged or replayed presentations of biometric data.

Face recognition is one of the most widely deployed biometric identification technologies, used in applications ranging from mobile device authentication and banking to border control and surveillance. Face recognition systems are vulnerable to presentation attacks, in which an attacker presents a photograph, a digital screen replay, or a three-dimensional mask in front of the camera instead of a live face. Face Anti-Spoofing is the discipline within biometric security that focuses on detecting these attacks and preventing unauthorized access. As presentation attacks grow more sophisticated, the development of robust and efficient anti-spoofing systems using deep learning has become a critical research priority.

1.1 MACHINE LEARNING

Machine Learning is a subset of Artificial Intelligence that focuses on developing systems capable of automatically learning patterns, relationships, and insights from data without explicit programming. Instead of being told exactly how to perform a task, the systems use algorithms to analyze data, identify trends, and improve their performance over time through experience, enabling computers to learn from past data and make intelligent decisions or predictions for new, unseen data.

The algorithms are trained using historical datasets and can perform tasks such as prediction, classification, clustering, and decision making. The more data the system is exposed to, the better it becomes at recognizing complex patterns, much like how humans learn from experience. In anti-spoofing applications, Machine Learning algorithms learn to distinguish the visual characteristics of genuine live faces from those of spoofed presentations by training on labeled examples of both categories.

Machine Learning can be broadly classified into three main categories:

I. Supervised Learning: The algorithm is trained on a labeled dataset, meaning each training example is paired with the correct output. The system learns the mapping between input features and output labels and predicts outcomes for new data. Anti-spoofing classifiers that learn from labeled live and spoof image datasets are examples of supervised learning.

II. Unsupervised Learning: The data is unlabeled and the algorithm explores the underlying structure or patterns within the dataset without

explicit guidance. It groups similar data points into clusters or reduces the dataset's dimensionality. In anti-spoofing research, unsupervised methods are used to discover latent domain structure across different benchmark datasets.

III. Reinforcement Learning: The model is trained through trial and error, receiving rewards or penalties for its actions. Over time, the system learns to maximize cumulative rewards. Reinforcement learning has been explored in adaptive anti-spoofing systems that adjust detection thresholds based on environmental feedback.

1.2 DEEP LEARNING

Deep Learning is an advanced branch of Machine Learning that enables computers to automatically learn complex features and patterns from large datasets. It is inspired by the structure of the human brain, in which multiple layers of interconnected neurons process and extract information. Deep Learning uses Artificial Neural Networks to analyze and interpret raw data without manual feature extraction, and its ability to learn directly from data has transformed fields like computer vision, speech recognition, natural language processing, and biometric security.

Deep Learning relies on neurons arranged in layers. Each neuron processes inputs, applies mathematical operations including weighted sums and activation functions, and passes the result forward. During training, the model fine-tunes these weights through a process called backpropagation, minimizing prediction errors and improving accuracy over successive training iterations.

The typical architecture of a Deep Learning model includes:

I. Input Layer: Accepts raw data such as the pixel values of a video frame or the frequency spectrum of a face image.

II. Hidden Layers: Multiple intermediate layers that progressively extract and combine features from low-level patterns such as edges and textures to high-level semantic representations such as spoofing artifacts and identity characteristics.

III. Output Layer: Produces the final prediction or classification result, such as a binary live-or-spoof probability score used to trigger an access decision.

Deep Learning models require large datasets and high computational power. Advanced hardware such as Graphics Processing Units and Tensor Processing Units enables faster training across millions of examples. Optimization algorithms such as Adam, Stochastic Gradient Descent, and RMSprop help fine-tune model parameters to achieve high classification accuracy and strong generalization to unseen data.

1.3 COMPUTER VISION

Computer Vision is a field of Artificial Intelligence that enables machines to interpret and understand visual information from the world, such as images and video streams. It combines image processing, pattern recognition, and deep learning to extract meaningful information from raw pixel data. Computer Vision systems can perform tasks such as object detection, image classification, facial recognition, motion estimation, and scene understanding, making them fundamental to modern biometric and security applications.

In face anti-spoofing, Computer Vision provides the core techniques used to capture, process, and analyze facial images from video streams. Face detection algorithms localize the face region within each video frame, enabling the extraction of a region of interest for downstream classification. Image normalization, resizing, and color space conversion prepare the extracted face regions for neural network inference. Optical flow computation, which estimates the apparent motion of pixels between consecutive frames, provides temporal information that distinguishes the natural micro-movements of a live face from the complete stillness of a printed photograph or rigid mask.

Convolutional Neural Networks are the dominant deep learning architecture in Computer Vision. They apply learned convolutional filters to input images to extract spatial feature maps that capture texture, edges, and structural patterns at multiple scales. Models such as MobileNetV3 and EfficientNet apply depthwise separable convolutions and compound scaling to achieve high classification accuracy with reduced computational cost, making them suitable for real-time anti-spoofing deployment. Frequency domain analysis extends spatial Computer Vision techniques by transforming images into the frequency domain using the Fast Fourier Transform, enabling the detection of periodic noise patterns such as Moire fringes that are invisible in the spatial domain but reveal digital screen replay attacks.

1.4 FACE ANTI-SPOOFING

Face Anti-Spoofing is a critical security component in biometric authentication systems that detects attempts to deceive a face recognition system using a fake or replayed facial presentation. As face recognition technology has been deployed in high-security applications such as mobile payments, border

control, and physical access management, the threat of presentation attacks has grown significantly. A presentation attack occurs when an adversary presents an artefact such as a printed photograph, a digital screen displaying a recorded video, or a silicone face mask in front of the camera to impersonate a legitimate user.

Detecting presentation attacks is challenging because modern attack materials have improved to closely resemble the visual appearance of a real face. High-resolution photo prints, OLED screen replays, and professionally manufactured silicone masks can fool naive RGB-based liveness checks. Additionally, real-world deployment introduces variability in lighting conditions, camera quality, subject distance, and facial orientation, all of which can reduce the reliability of a single-model detection approach. A robust anti-spoofing system must maintain high detection accuracy across all these conditions while operating within the latency constraints of a real-time authentication pipeline.

Anti-spoofing system addresses these challenges through a three-stage cascaded inference pipeline. The first stage uses MobileNetV3 for fast single-frame screening and immediately resolves clearly live faces above a 95% confidence threshold. The second stage applies ForensicNet, a dual-branch architecture combining EfficientNet spatial texture analysis with FFT-based frequency domain inspection, to detect subtle attack artifacts in ambiguous cases. The third stage performs temporal motion verification over a 16-frame buffer to reject static photograph attacks that exhibit zero frame-to-frame motion. This cascaded design achieves both high detection accuracy and real-time throughput, making it suitable for practical deployment in access control and authentication systems.

Domain generalization is a further challenge in face anti-spoofing, as models trained on one benchmark dataset often fail to generalize to unseen datasets due to differences in camera sensor, lighting, and image compression. The DiffFAS framework addresses domain shift by separating it into image quality shift and image style shift. A diffusion-based generative model performs live-to-spoof transformation that enriches the training set with cross-domain attack samples, while a Sample-Level Relative Quality loss integrates BRISQUE-based image quality scores into the training objective to direct the classifier towards high-quality spoof evidence and reduce sensitivity to degraded or low-quality inputs.

1.5 SUMMARY

Machine Learning, Deep Learning, Computer Vision, and Face Anti-Spoofing are the key technologies underlying the proposed AntiSpoofing Detection System. Machine Learning enables systems to learn discriminative patterns from labeled data and generalize to new examples without explicit programming. Deep Learning, through multi-layer neural networks, automatically extracts complex visual features from raw image data, removing the need for handcrafted feature engineering and enabling high-accuracy classification of live and spoofed faces.

Computer Vision provides the image processing, face detection, and optical flow techniques that prepare raw video input for neural inference and enable temporal motion analysis across frame sequences. Face Anti-Spoofing combines these foundational technologies within a security-focused pipeline designed to detect print attacks, screen replay attacks, and static photograph attacks in real time.

Together, these technologies make the proposed anti-spoofing system accurate, efficient, and robust against diverse attack types and deployment environments. The cascaded multi-stage pipeline, domain-generalized training through diffusion-based data augmentation.

CHAPTER 2
SURVEY ON ANTISPOOFING DETECTION

The use of deep learning for face presentation attack detection has grown rapidly over the past decade, driven by the widespread adoption of facial recognition in high-security applications such as mobile payments, border control, and access management. This growth has been especially significant in countering increasingly sophisticated spoofing attacks that use printed photographs, digital screen replays, and three-dimensional face masks. This review examines current research on face anti-spoofing systems, deep learning architectures employed for liveness detection, frequency domain analysis techniques, temporal modeling methods, and domain generalization approaches. The review identifies the key limitations of existing methods and explains how the proposed multi-stage pipeline overcomes them.

Maatta et al. (2011) introduced one of the earliest automated face anti-spoofing methods by combining Local Binary Pattern descriptors with Support Vector Machine classification. Their three-level LBP analysis extracted texture features at multiple granularities from face images and achieved 90.0% accuracy on the NUAA photograph impersonation database. The LBP-SVM baseline established a foundational framework for texture-based anti-spoofing that

influenced numerous subsequent studies. However, the method was designed for print attack detection only and did not generalize to video replay attacks. The handcrafted feature pipeline also lacked the capacity to automatically learn discriminative representations, requiring domain experts to manually select and validate relevant texture descriptors for each new attack type.

Boulkenafet et al. (2017) presented a face anti-spoofing method based on color texture analysis across multiple color spaces including YCbCr and HSV in addition to the standard RGB representation. Their approach used Local Binary Pattern descriptors extracted from multiple color channels and fused the resulting feature vectors using a linear SVM classifier. The multi-color-space texture method achieved 90.1% accuracy on the OULU-NPU dataset and demonstrated improved robustness to illumination changes compared to single-channel analysis. However, the method relied entirely on handcrafted feature extraction and required careful parameter selection for each new dataset. The approach also failed to detect high-quality silicone mask attacks and showed significant performance drops on video compressed with lossy codecs.

Atoum et al. (2017) proposed a patch-based Convolutional Neural Network approach for face anti-spoofing that processed local facial regions independently before aggregating predictions through majority voting. Their method achieved 95.2% accuracy by training the model to detect local spoofing artifacts rather than requiring global face understanding. Patch-level training allowed the system to handle partially occluded faces and varying face sizes more robustly than image-level classifiers. However, the patch aggregation strategy introduced inconsistencies when different patches produced contradictory predictions, particularly for high-resolution screen attacks where only specific facial regions exhibited detectable frequency artifacts. The majority voting

mechanism also discarded confidence information, preventing calibrated probability output for downstream risk assessment.

Tan et al. (2019) introduced EfficientNet, a family of convolutional neural networks that achieved state-of-the-art accuracy through compound scaling of network depth, width, and input resolution simultaneously. EfficientNet-B4 achieved 83.0% top-1 accuracy on ImageNet with only 19 million parameters, outperforming previous architectures with far fewer computational resources. The compound scaling method demonstrated that simultaneously increasing all three network dimensions according to a fixed scaling coefficient consistently produced better accuracy-efficiency tradeoffs than scaling any single dimension alone. EfficientNet architectures have subsequently been widely adopted in medical and biometric applications due to their favorable accuracy-to-parameter ratio, which is particularly relevant for deployment scenarios where both accuracy and computational cost are constrained.

Howard et al. (2019) introduced MobileNetV3, a highly optimized lightweight convolutional neural network designed for mobile and edge deployment. Their compound neural architecture search combined hard swish activation functions with squeeze-and-excitation optimization to achieve 75.2% top-1 accuracy on ImageNet with only 5.4 million parameters. MobileNetV3's depthwise separable convolutions reduced the number of multiply-accumulate operations by a factor of eight compared to standard convolutions, enabling real-time inference on CPU-only hardware. These efficiency characteristics make MobileNetV3 particularly suitable as a fast screening stage in multi-stage anti-spoofing pipelines, where the goal is to rapidly classify easy cases while conserving computation for genuinely ambiguous inputs.

Yang et al. (2019) conducted a comprehensive survey of face anti-spoofing methods, categorizing existing approaches by their feature modality including texture-based, frequency-based, temporal, and depth-based methods. Their analysis revealed that texture-based CNN classifiers consistently outperformed handcrafted feature approaches but struggled with cross-dataset generalization. Frequency-domain methods were found to be particularly effective against screen-based replay attacks due to their sensitivity to Moire patterns and periodic noise. Temporal methods using optical flow and video analysis showed strong performance against print attacks but required multi-frame processing latency. The survey concluded that hybrid approaches combining multiple feature modalities were most likely to achieve robust performance across diverse attack types.

Shao et al. (2021) introduced a multi-adversarial discriminative deep domain generalization framework for anti-spoofing. Their method trained a domain-invariant feature extractor across multiple source domains using adversarial training to improve generalization to unseen target domains. The framework achieved state-of-the-art cross-dataset performance with an HTER of 7.3% under leave-one-out evaluation across four major anti-spoofing benchmarks. However, the adversarial training procedure required simultaneous optimization of multiple competing objectives, making it unstable and difficult to converge reliably. The training process also required access to data from multiple source domains, limiting applicability in scenarios where only a single domain dataset is available.

Zhang et al. (2020) proposed a face anti-spoofing framework based on central difference convolution, which captures not only intensity information but also gradient information of local pixel differences. Their central difference

convolutional network achieved 96.7% accuracy on the CASIA-FASD benchmark and demonstrated improved robustness to illumination changes compared to standard convolutional architectures. The gradient-aware convolution operation allowed the network to detect subtle texture differences between live skin and printed or displayed face images. However, the central difference computation increased the number of convolutional operations, raising inference time by approximately 30% compared to standard convolutions without proportional accuracy gains across all benchmark datasets.

Li et al. (2022) proposed a frequency domain analysis method for face anti-spoofing that applied discrete Fourier transform processing to extract Moire patterns and periodic artifacts characteristic of screen replay attacks. Their approach achieved 97.1% detection accuracy on the Replay-Attack dataset by identifying high-frequency noise bands invisible in the spatial domain. The frequency domain analysis was particularly effective against high-resolution OLED screen replays, which closely mimic the spatial texture of real skin. Nevertheless, the method was specifically optimized for screen-based attacks and performed poorly against printed photograph attacks, achieving only 74.3% accuracy on print attack benchmarks. This single-modality limitation demonstrated the need for combining spatial and frequency analysis within a unified detection framework.

Wang et al. (2022) developed a multi-scale feature fusion network for presentation attack detection that combined shallow texture features with deep semantic representations. Their architecture processed face images at three different scales and fused the resulting feature maps through a learnable attention module, reaching 94.8% accuracy on the SiW-M dataset which includes diverse attack types. However, the multi-scale fusion strategy significantly increased

model size and memory footprint, and the system required careful calibration of scale weights during training. The inference time of 150 milliseconds per frame also limited its applicability in strict real-time scenarios requiring throughput above 15 frames per second.

Sun et al. (2021) proposed a temporal modeling approach for face anti-spoofing that captured motion dynamics across sequential video frames using a two-stream architecture combining a spatial stream with a temporal optical flow stream. Their method achieved 93.5% accuracy on the CASIA-FASD dataset by detecting the absence of natural facial micro-motions in printed photograph attacks. The optical flow stream effectively distinguished static spoofing materials from live faces exhibiting continuous micro-movements. However, the system required at least 16 consecutive frames for reliable optical flow estimation, introducing latency during initial face enrollment, and the two-stream design doubled the computational requirements compared to single-frame approaches.

Ge et al. (2024) proposed DiffFAS, the first diffusion model-based framework for face anti-spoofing, which addressed domain shift by separating it into two components: image quality shift and image style shift. The framework used a UNet-based diffusion network with a Spoofing Style Fusion Module to generate high-fidelity cross-domain and cross-attack spoof samples from live face images, enriching the training set without requiring additional physical attack collection. A Sample-Level Relative Quality loss integrated BRISQUE-based image quality scores into the training objective to direct the classifier towards high-quality spoof evidence. The method achieved state-of-the-art performance with an average HTER of 6.86% on the OCIM cross-domain

protocol and an average ACER of 6.68% on the WMCA unseen attack protocol, outperforming all prior generalization methods.

The overall limitations of the existing systems are:

I. Single-Modality Detection — Most existing methods rely solely on spatial texture features and cannot simultaneously capture both spatial and frequency domain evidence, limiting their detection coverage across diverse attack types.

II. Limited Cross-Domain Generalization — Models trained on one benchmark dataset fail to maintain performance on unseen datasets due to differences in camera sensor, lighting, and image compression characteristics.

III. High Computational Cost — Multi-scale fusion networks and vision transformer-based architectures achieve strong accuracy but exceed the latency constraints required for real-time 30 FPS deployment on commodity hardware.

IV. Single-Frame Dependency — Most classifiers evaluate each frame independently without temporal context, making them unable to detect static photograph attacks that are visually indistinguishable from live faces in a single frame.

V. Noise and Blur Sensitivity — Existing methods do not filter out motion-blurred or low-quality frames before inference, causing degraded frames to produce unreliable predictions that increase the false acceptance rate.

VI. Absence of Quality-Aware Training — Training pipelines treat all samples equally regardless of their image quality, causing the model to overfit to blurry or degraded spoof cues rather than learning from high-quality discriminative evidence.

VII. GAN and VAE Generation Limitations — Prior generative methods based on GANs and VAEs suffer from texture distortion, identity erosion, and overfitting to limited training data, reducing the usefulness of synthetically generated spoof samples.

VIII. Limited Attack Type Coverage — Existing datasets lack sufficient labeled samples for novel attack types such as flexible masks and partial face edits, preventing models from learning to detect emerging presentation attack categories.

IX. Lack of Visual Explainability —Most deep anti-spoofing models operate as black boxes and do not provide visual explanations of their decisions, reducing operator trust and making it difficult to validate or audit system behavior.

X. No Cascaded Efficiency Mechanism — Existing pipelines apply the same deep model uniformly to every input frame, wasting computation on clearly live faces that could be resolved by a lightweight fast-pass classifier.

2.1 SUMMARY

In conclusion, past studies made strong progress in texture-based anti-spoofing, frequency domain analysis, temporal motion modeling, and domain generalization for face presentation attack detection, but still face issues such as

poor cross-dataset generalization, single-modality detection coverage, high computational cost, and vulnerability to static photograph attacks. Spatial texture methods offer interpretable feature representations but miss frequency artifacts invisible in the spatial domain. Frequency domain methods detect screen replay attacks effectively but fail against print attacks. Temporal methods provide strong static attack rejection but introduce latency and double computational requirements.

Most systems also apply uniform inference to every input frame without a fast-pass mechanism, wasting computation on clearly live faces. The proposed AntiSpoofing Detection System overcomes these gaps through a three-stage cascaded pipeline that combines MobileNetV3 fast screening, ForensicNet dual-branch spatial and frequency analysis, and 16-frame temporal motion verification. Domain generalization is addressed through DiffFAS diffusion-based data augmentation and Sample-Level Relative Quality loss training, achieving robust and accurate liveness detection across diverse attack types, camera conditions, and benchmark datasets.

CHAPTER 3

ANTISPOOFING DETECTION USING DIFFUSION-BASED GENERATIVE FRAMEWORK AND QUALITY-AWARE LEARNING

3.1 OVERVIEW OF DIFFFICULTY ANALYSIS AND DOMAIN SHIFT

Face anti-spoofing systems face the challenge of domain shift, which impacts the generalization performance of existing methods when deployed across different datasets or environments. Domain shift is deconstructed into two separable factors: image style and image quality. Image quality influences the purity of the presentation of spoof information, while image style affects the manner in which spoof information is presented. Both quality and style introduce erroneous prior information to the network during training, thereby posing a formidable obstacle in learning precise spoof cues that generalize reliably across source and target domains.

Image quality is quantified by factors such as image degradation, blur, and low resolution, and its effect on anti-spoofing can be measured using no-reference quality assessment metrics. Image style is characterized by local image statistics, reflecting the color and texture differences between images of the same

or different types of attacks captured across different benchmark datasets. Existing domain generalization methods address domain shifts by incorporating human prior information such as domain-specific details and identity information, which are considered disturbances that should be disregarded. The DiffFAS framework instead employs image quality as a quantifiable metric to directly counter quality shift, and applies generative methods to address the unquantifiable image style shift component.

For face anti-spoofing tasks, live faces can be easily collected, but acquiring spoof faces is time-consuming and labor-intensive. The DiffFAS framework therefore performs live-to-spoof transformation to generate a large number of spoof samples from live face images. The training objective is defined as the reconstruction process of spoof images rather than unconditional generation, which allows the network to directly learn spoof texture patterns and avoid the texture degradation that occurs when diffusion models suppress high-frequency information during standard denoising. This design choice also mitigates overfitting, which is a common challenge encountered by prior GAN-based and VAE-based generative methods that produce spoof images too closely resembling the training set.

3.2 DATA ACQUISITION

The DiffFAS framework is trained and evaluated using multiple publicly available face anti-spoofing benchmark datasets including OULU-NPU, CASIA-MFSD, Idiap Replay-Attack, and MSU-MFSD, which contain various styles of print and replay attack, and WMCA and PADISI, which contain multiple types of novel attacks such as flexible masks and glasses. For the classification experiments, input images are cropped using RetinaFace face detection and

resized to 256x256 pixels. The MobileViT architecture is used as the classification backbone. For the OCIM cross-domain experiments, additional trials using ResNet-18 are conducted to ensure fair comparison with prior published methods.

The training data is organized into a Spoofing Style Pool and Live-Spoof Pair Images. The Spoofing Style Pool consists of spoof face images from multiple domains used as guide images to provide the target spoofing texture style during generation. The Live-Spoof Pair Images consist of matched live and spoof face images from the same subject used to train the UNet diffusion network on the spoof image reconstruction objective. Only data from visible domains or attack types are used during generation to prevent data leakage in cross-domain and cross-attack evaluation protocols.

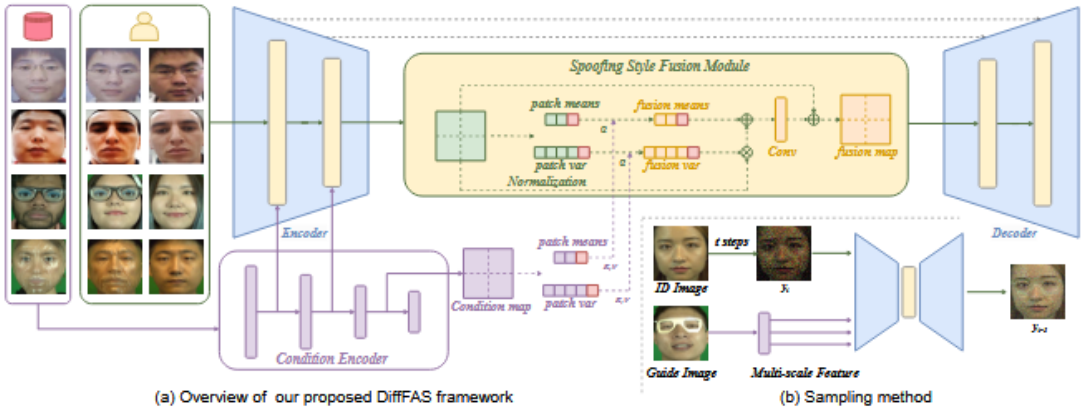


Figure 3.1 DiffFAS Framework— UNet Architecture

Figure 3.1 illustrates the proposed DiffFAS generative framework. The system is a UNet-based network composed of a Spoofing Style Encoder and a noise prediction module. The Spoofing Style Encoder extracts texture features from a randomly selected guide spoof image and obtains multi-scale feature maps from different encoder layers. The Spoofing Style Fusion Module is applied

within the UNet to reduce the introduction of identity information from the conditional branch and to achieve information aggregation with the backbone through cross-attention. This allows the network to fully capture the spoof texture and achieve high-fidelity spoof synthesis. During the inference stage, image editing techniques are employed for the sampling process to improve consistency between the generated identity and the original live face identity.

3.3 DIFFFICULTY-BASED GENERATIVE FRAMEWORK

3.3.1 Diffusion Model Preliminaries

Diffusion models are generative models trained to predict an image from random noise through a gradual denoising process consisting of a forward process and a reverse process. The forward process progressively adds Gaussian noise to the original image over T timesteps according to a fixed variance schedule. The true reverse posterior is approximated by training a UNet model to predict the initial noise using mean squared error loss. In the reverse process, the model samples from Gaussian noise by iteratively applying the learned denoising function to reconstruct the original image.

$$q(y_t \mid y_{t-1}) = N(y_t ; \sqrt{(1-\beta_t)} \cdot y_{t-1}, \beta_t \cdot I) \quad \text{Eq 3.1}$$

$$L_{mse} = E \, || \varepsilon - \varepsilon_\theta(y_t, t) ||^2 \quad \text{Eq 3.2}$$

$$p_\theta(y_{t-1} \mid y_t) = N(y_{t-1} ; \varepsilon_\theta(y_t, t)) \quad \text{Eq 3.3}$$

Equation 3.1 describes the forward diffusion process that progressively corrupts the original image into Gaussian noise. Equation 3.2 is the MSE training objective used to train the UNet noise predictor. Equation 3.3 describes the reverse denoising process used to reconstruct the image from noise. The DiffFAS framework redefines the training objective as spoof image reconstruction,

replacing the standard unconditional generation goal to allow the network to directly learn spoof texture patterns from training data.

3.3.2 Generation Objective

The generation objective is to transform a live face image carrying a known identity into a high-fidelity spoof version of that same identity while adopting the spoofing style from a randomly selected guide spoof image. Given a live face image and a guide spoof image from any domain, the DiffFAS network produces a generated spoof image that carries the identity of the live face but displays the surface texture and attack characteristics of the guide spoof image. During training, same-domain image pairs are used for efficiency since cross-domain pairs are scarce. The forward diffusion process corrupts the spoof image with noise, and the reverse process reconstructs the spoof image conditioned simultaneously on the live face for identity and on the guide spoof image for style.

$$X_t = \sqrt{(\bar{\alpha}_t)} \cdot X_s^{id1} + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon \qquad \text{Eq 3.4}$$

$$p_\theta(X_{t-1}|X_t) = N(X_{t-1} ; \varepsilon_\theta((X_t, X_l^{id1}), t, X_s^{id2})) \qquad \text{Eq 3.5}$$

Equation 3.4 defines the forward diffusion process applied to the spoof image, where the live face identity image is concatenated with the noisy spoof image as UNet input. This concatenation operation aligns the reconstructed spoof image with the corresponding live face image during training, maintaining identity consistency. Equation 3.5 describes the conditioned reverse denoising process where the UNet is guided by both the live face identity and the guide spoof style simultaneously.

.

3.3.3 Spoofing Style Fusion Module

The Spoofing Style Fusion Module processes the condition feature maps from the Condition Encoder into arrays of patch-level means and variances. Parallel operations are executed on the feature maps derived from the UNet backbone to produce a corresponding set of means and variances. Cross-attention is then performed between the condition statistics and the backbone statistics to produce fused means and fused variances. The fused statistics are resized and upsampled using bilinear interpolation to match the feature map spatial dimensions, and then combined with the original backbone feature map through a normalization-based injection operation. The STFM module is applied to all UNet blocks with spatial dimensions at or below 32x32.

$$\mu_f = attn(Q(\mu_x), KV(\mu_cond)) \qquad \textit{Eq 3.6}$$

$$\sigma_f = attn(Q(\sigma_x), KV(\sigma_cond)) \qquad \textit{Eq 3.7}$$

$$X_f = X + conv(norm(X) \cdot \sigma_f + \mu_f) \qquad \textit{Eq 3.8}$$

Equations 3.6 and 3.7 perform cross-attention between the UNet backbone statistics and the condition encoder statistics for the means and variances respectively. Equation 3.8 combines the fused statistics with the backbone feature map through a convolutional normalization operation, injecting the spoofing style into the reconstruction process while preserving the structural identity of the live face.

3.4 INFERENCE AND QUALITY-AWARE CLASSIFICATION

3.4.1 Disentangled Guidance

During inference, the DiffFAS framework uses classifier-free guidance to control the intensity of the spoofing style transfer. The spoofing style condition

is set to zero with a certain probability during training to enable the model to operate both conditionally and unconditionally. During the sampling phase, the denoising outputs for both conditional and unconditional scenarios are computed independently at each timestep. The difference between these outputs yields the specific influence of the guidance, and a guidance strength parameter gamma controls the extent to which the spoofing style is applied to the generated image.

$$p_\theta(X_{t-1}|X_t) = N(\varepsilon_\theta((X_t, X_l), t)) + \gamma\cdot(N(\varepsilon_\theta((X_t, X_l), t, X_{s_guide})) - N(\varepsilon_\theta((X_t, X_l), t))) \quad \text{Eq 3.9}$$

Equation 3.9 describes the classifier-free guidance sampling process. Increasing the parameter gamma produces more pronounced spoofing texture in the generated image, while decreasing gamma preserves more of the live face's original appearance. This capability allows the generation of training samples with varying levels of spoof strength, thereby enhancing the robustness of the downstream classification network.

3.4.2 Image Editing Based Sampling

To improve identity consistency and reduce the impact of mismatched identity pairs in training data, the DiffFAS inference stage employs an image editing-based sampling approach. Instead of sampling directly from pure Gaussian noise, the framework introduces a limited number of forward diffusion steps to the original live face image, producing a partially noised version that retains the structural identity of the face. Conditional denoising is then performed from this noised live image rather than from random Gaussian noise. This approach allows the generated spoof image to maintain closer identity alignment with the original live face while still acquiring the spoofing texture from the guide image

.

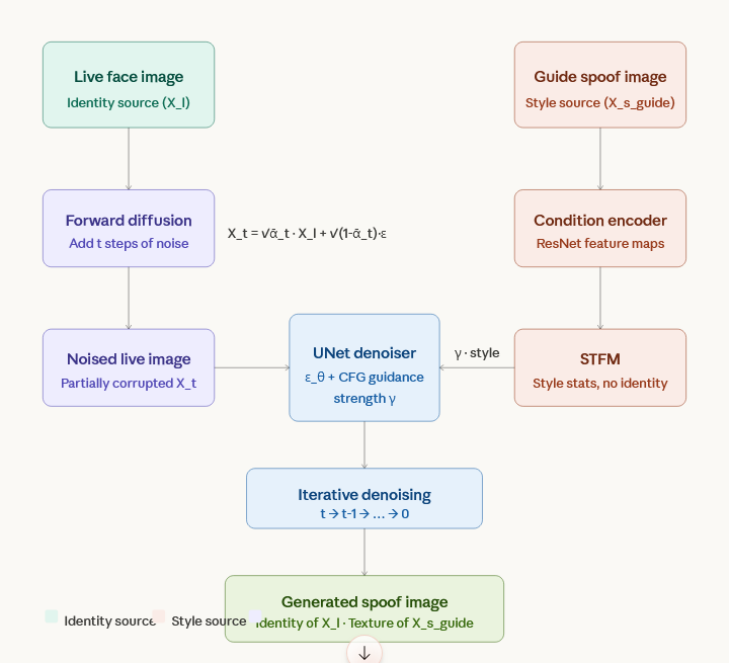


Figure 3.2 DiffFAS Inference Sampling Process

Figure 3.2 illustrates the image editing-based inference sampling process. The live face identity image is partially corrupted by t steps of forward diffusion noise. Conditional denoising is then applied from the noised live image using classifier-free guidance with the guide spoof image as the style condition. The resulting generated image retains the identity structure of the live face while displaying the attack texture of the guide spoof image across the target domain or attack type.

3.4.3 Sample-Level Relative Quality Loss

As a primary manifestation of domain shifts, furnishing the network with quality prior information can significantly improve its performance. The Sample-Level Relative Quality loss classifies data based on specific spoofing styles rather

than simple binary classification. Within each spoofing style class, margin softmax loss is employed to learn compact clusters, following approaches similar to CosFace and ArcFace that introduce angular and additive margins to the classifier. The BRISQUE no-reference image quality score, which is inversely related to image quality, is integrated into the margin computation to control network attention towards high-quality samples.

The absolute BRISQUE quality scores for a training batch are converted into a standard normal distribution and scaled according to the 3-sigma rule to ensure most samples fall within a defined interval. The resulting normalized quality scores are clipped to a range centered at zero, producing per-sample relative quality scores that are directly proportional to image quality. High-quality images receive a larger additive margin, requiring the network to produce more confident and well-separated predictions for them, while low-quality images receive a reduced margin to prevent overfitting to noisy or degraded spoof cues.

$$b_norm = (b_mean - b_aq) / (b_std + eps) \qquad \text{Eq 3.10}$$

$$b_rq = clip(b_norm / 3,\ -n1/N,\ n2/N) \qquad \text{Eq 3.11}$$

$$\psi = -s \cdot b_rq ,\quad \omega = s \cdot (1 + b_rq) \qquad \text{Eq 3.12}$$

$$g(\theta) = \cos(\theta - s \cdot b_rq) - s \cdot (1 + b_rq) \qquad \text{Eq 3.13}$$

$$L_RQ = -(1/n) \Sigma \log [e^{(m \cdot g(\theta))} / (e^{(m \cdot g(\theta))} + \Sigma e^{\cos(\theta_j)})] \qquad \text{Eq 3.14}$$

Equation 3.10 normalizes the absolute BRISQUE scores within the batch into a standard distribution. Equation 3.11 computes the relative quality score for each sample by clipping the normalized score to a range determined by the proportions of above-average and below-average quality samples in the batch.

Equations 3.12 and 3.13 define the quality-adaptive angular margin and additive margin functions respectively. Equation 3.14 is the final Relative Quality loss that applies the adaptive margin to the softmax classification objective, directing the network to extract discriminative spoof cues primarily from high-quality training samples. Different scaling factors are applied to live and spoof clusters to ensure that the live class cluster remains more compact than the spoof cluster.

3.5 OUTCOME AND OBSERVATIONS

3.5.1 Generative Experiment Results

The DiffFAS generative framework demonstrates strong cross-attack and cross-domain spoof generation capabilities across all evaluated benchmark datasets. For mask attacks in the PADISI dataset, the model learns specific texture features from masks with high visual homogeneity in the training set and replaces live skin texture with the learned attack texture while maintaining the identity of the original live face. This capability enriches mask diversity in the training set without requiring additional physical mask collection. For face-edit attacks, the model performs partial replacement of only the edited facial region without modifying the overall face structure, demonstrating fine-grained texture transfer capability. The model achieves robust texture learning even when trained on as few as 20 to 30 image pairs per attack type.

3.5.2 Cross-Attack Results On Wmca

On the WMCA unseen attack protocol which includes seven novel attack types not present in the training data, DiffFAS-V achieves an average ACER of 6.68%, improving upon the best prior baseline by 4.4 percentage points. The most significant improvements are observed for flexible mask attacks and replay attacks, where the generative framework successfully synthesizes representative

training samples even though the specific attack type was not available during training. The Relative Quality loss contributes additional generalization by preventing the classifier from anchoring its decision boundary on low-quality or blurry spoof cues that do not generalize across attack types.

3.5.3 Ablation Study

Ablation experiments on the STFM design confirm that the asymmetric patch size strategy is critical for balancing identity preservation and spoofing texture transfer. Excessively small means patch sizes lead to significant alterations in facial identity, such as the guide image's identity blending into the generated face. Larger variance patch sizes reduce fine-grained texture detail, losing attack-specific surface characteristics. The recommended configuration uses a means patch size of 6x6 and a variance patch size of 2x2 to achieve the optimal balance between identity and spoofing texture representation in the generated samples. Ablation of the inference hyperparameters confirms that increasing forward timesteps or guidance coefficient gamma increases the intensity of the spoof texture while reducing identity preservation.

Ablation of framework components confirms that both the DiffFAS generative augmentation and the Sample-Level Relative Quality loss contribute independently and complementarily to the overall performance. Removing the generative component reduces the average HTER on the MI-to-C protocol from 15.06% to 15.50%, and removing the quality loss increases it to 16.91%. Using MobileViT alone without either component results in 24.57% HTER, confirming that both proposed contributions are essential for the observed performance gains.

3.5.4 Limitations

The DiffFAS framework currently faces a constraint in open-world face reconstruction. The generative model requires a corresponding identity to be present in the training set in order to generate its spoof version, limiting its ability to produce spoof samples for entirely new identities not seen during training. Enhancing the open-world generation capability is identified as a key direction for future research. The forward diffusion process is also computationally intensive, with inference requiring multiple denoising steps per generated sample, which increases the time required to produce large-scale augmented training datasets.

The Sample-Level Relative Quality loss depends on BRISQUE scores as the quality measure, which is a no-reference metric designed for general image quality assessment. BRISQUE may not correctly quantify quality degradation introduced by specific attack-related artifacts such as the surface reflectance of silicone masks or the color banding patterns of printed attacks, limiting the precision of the quality-adaptive margin in these cases. Additionally, the system operates on RGB image inputs only and does not exploit depth or near-infrared sensor data that could provide additional discriminative information for detecting high-quality physical mask attacks.

3.6 SUMMARY

The DiffFAS framework addresses the two primary sources of cross-dataset performance degradation in face anti-spoofing by separately handling image quality shift and image style shift. The UNet-based diffusion model with the Spoofing Style Fusion Module generates identity-consistent cross-domain and cross-attack spoof samples from live face images, enriching the training set

without requiring additional physical attack data collection. The image editing-based sampling approach and classifier-free guidance enable controlled generation of spoofing textures with adjustable strength while maintaining live face identity consistency.

The Sample-Level Relative Quality loss integrates BRISQUE-based quality scores into the training objective through per-sample adaptive angular and additive margins, directing the classification network to extract discriminative spoof cues from high-quality training samples while reducing sensitivity to degraded or blurry inputs. Together, these two components achieve state-of-the-art cross-domain anti-spoofing performance with an average HTER of 6.86% on the OCIM protocol and an average ACER of 6.68% on the WMCA unseen attack protocol, establishing a strong foundation for generalizable face liveness detection in real-world deployment scenarios.

CHAPTER 4

A MULTI-STAGE DEEP LEARNING PIPELINE FOR ANTISPOOFING DETECTION

4.1 ANTISPOOFING DETECTION

A cascaded multi-stage deep learning pipeline forms the core of the AntiSpoofing Detection System, combining a fast lightweight screening model, a dual-branch forensic analysis network, and a temporal motion verification stage to detect face presentation attacks in real-time video streams. Each stage of the pipeline targets a different class of spoofing attack and is invoked only when the previous stage cannot make a confident classification, ensuring that the computational cost of deep forensic analysis is reserved for genuinely ambiguous cases. This modular design enables the system to maintain real-time throughput at 30 frames per second while still applying rigorous forensic checks to inputs that cannot be resolved by the lightweight first stage alone.

The pipeline is organized into four sequential functional layers. The first layer handles face detection and region of interest extraction using Haar Cascade detection, resizing each extracted face crop to the required input resolution and storing it in a 16-frame rolling buffer for downstream temporal analysis. The second layer applies the fast screening model to each incoming face crop,

performing a rapid single-frame liveness assessment with a blur pre-filter. The third layer applies the dual-branch forensic analysis model to cases escalated from the fast screening stage, combining spatial texture analysis with frequency domain Moire detection. The fourth layer performs temporal motion verification over the buffered frame sequence to reject static photograph attacks that cannot be identified from spatial or frequency features alone.

Fast Screening Model (FastScreenNet): A lightweight MobileNetV3-based classifier that processes 128x128 pixel face crops. A Laplacian variance pre-filter discards motion-blurred frames before inference. Faces scoring above the 95% confidence threshold are immediately classified as live. Scores at or below 95% are escalated to the forensic stage.

Forensic Analysis Model (ForensicNet): A custom dual-branch architecture that processes 224x224 pixel face crops through two parallel streams. Branch A applies EfficientNet for spatial texture feature extraction. Branch B applies a learnable FFT convolutional layer for frequency domain Moire pattern detection. The combined output of both branches is fused and classified at a 50% threshold.

Temporal Verification Stage: A motion analysis layer that computes dense optical flow across the 16-frame rolling buffer. Natural live faces always exhibit non-zero micro-motion, while printed photographs remain completely static. A motion score of zero results in an immediate spoof classification regardless of the Stage 1 and Stage 2 outputs.

Final Output Renderer: A bounding box overlay rendered on the original video frame that communicates the classification result visually. A green box

indicates a confirmed live face that has passed all three stages. A red box indicates a spoof or attack presentation rejected at any stage of the cascade.

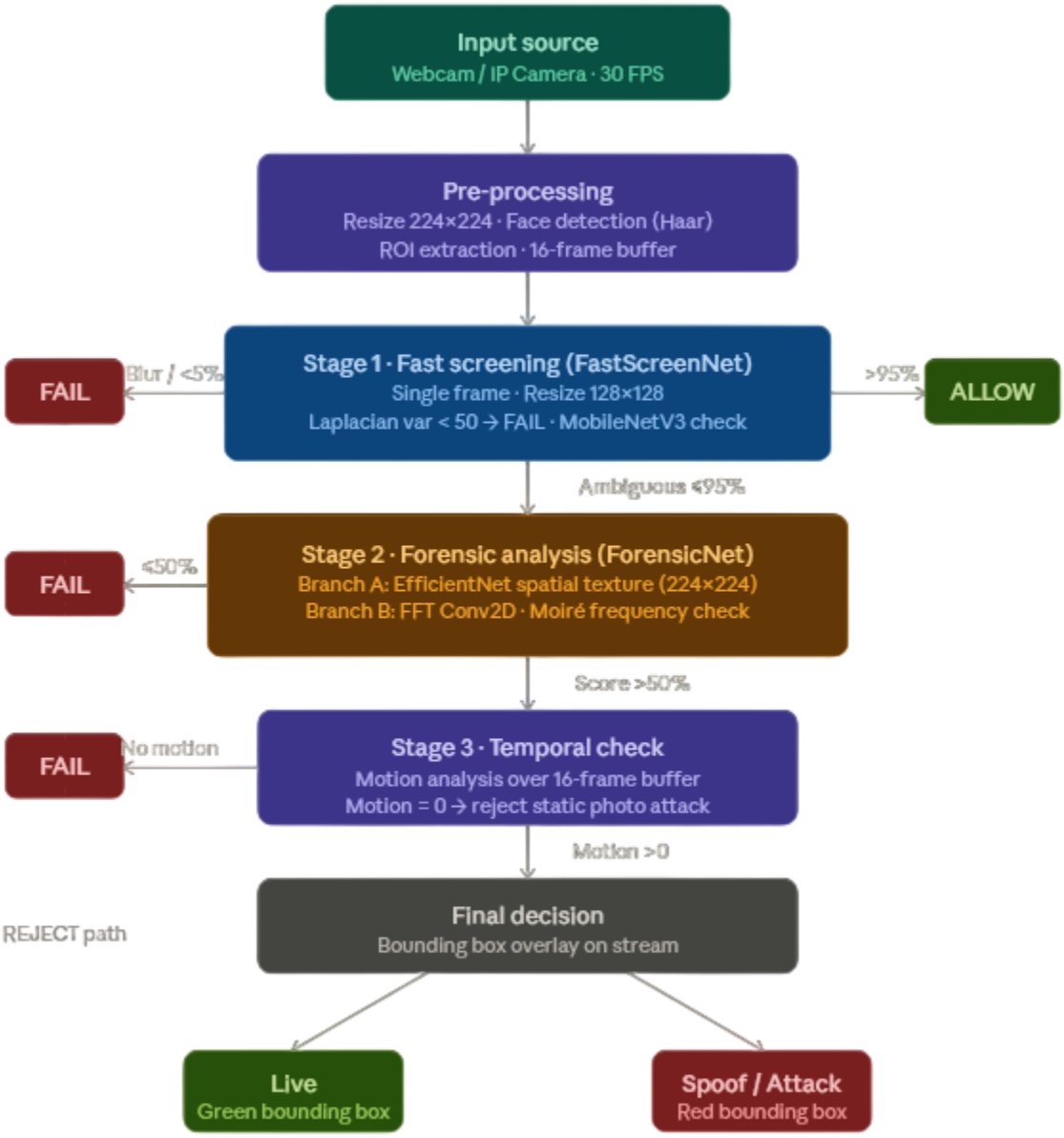


Figure 4.1. Workflow of the Multi-Stage Deep Learning Pipeline for AntiSpoofing Detection

The above Figure 4.1. illustrates the complete workflow of the AntiSpoofing Detection System from video frame capture through the three-stage cascade to the final bounding box output. The pipeline includes the Laplacian blur pre-filter, the fast screening decision gate at the 95% threshold, the dual-branch forensic analysis with the 50% decision threshold, and the 16-frame temporal motion verification stage. The green and red bounding box outputs are rendered in real time on the output video display.

4.2 DEEP LEARNING TECHNIQUES USED IN THE ANTISPOOFING SYSTEM

Transfer Learning: Pre-trained ImageNet weights initialize both the MobileNetV3 backbone in the fast screening model and the EfficientNet backbone in the forensic analysis model. Final classification heads are replaced and fine-tuned on anti-spoofing training data, enabling powerful general visual features to be efficiently adapted to the specific task of liveness classification with reduced training time.

Depthwise Separable Convolution: The fast screening model uses MobileNetV3's depthwise separable convolutions, which factorize each standard convolution into a depthwise spatial filter and a pointwise channel-mixing operation. This reduces the multiply-accumulate operation count by a factor of approximately eight, enabling real-time CPU inference at 30 FPS without GPU acceleration.

Squeeze-and-Excitation Optimization: Both MobileNetV3 in the fast screening stage and EfficientNet in the forensic analysis stage incorporate squeeze-and-excitation modules. These modules compute global channel statistics and apply learned per-channel scaling weights to recalibrate feature

responses, allowing the network to emphasize the most diagnostically relevant channels for liveness classification.

FFT-Based Frequency Domain Analysis: The forensic analysis Branch B applies a two-dimensional Fast Fourier Transform to the 224x224 face crop, transforming the spatial image into a frequency spectrum. Convolutional filters then detect structured periodic noise patterns including Moire fringes and aliasing artifacts introduced when a camera captures a face displayed on a digital screen, which are invisible in the spatial domain but clearly distinguishable in the frequency domain.

Laplacian Variance Blur Detection: A Laplacian variance pre-filter is applied before any neural inference. The variance of the Laplacian response map of each face crop is computed, and frames with variance below 50 are immediately discarded as motion-blurred, preventing degraded frames from generating unreliable liveness predictions at the fast screening stage.

Optical Flow Temporal Analysis: Dense optical flow is computed between consecutive frame pairs in the 16-frame rolling buffer. The mean flow magnitude within the face bounding box is aggregated across all frame pairs to produce a temporal motion score. This score distinguishes live faces exhibiting natural micro-motion from static photograph attacks that remain completely rigid across all frames in the buffer.\

4.3 WORKFLOW

I. Input and Pre-processing

Source: Webcam or IP Camera — RGB Stream at 30 FPS.

Resize: Each captured frame is resized to 224x224 pixels.

Face Detection: Haar Cascade frontal face detector localizes the face region within each frame.

ROI Extraction: The detected face region of interest is extracted from the full frame.

Buffer Management: The last 16 face crops are stored in a rolling buffer for temporal analysis in Stage 3.

$$Var(L(I)) = (1/N)\ \Sigma\ (L(I_k) - \mu_L)^2 \qquad Eq\ 4.1$$

Eq 4.1 computes the Laplacian variance of the face crop to assess motion blur before neural inference. L(I_k) is the Laplacian response at pixel k and μ_L is the mean response across all N pixels. Frames with variance below 50 are classified as motion-blurred and discarded, preventing degraded inputs from entering the fast screening model.

II. Stage 1 — Fast Screening (The Gatekeeper)

Stage 1 applies a single frame liveness check on a face crop resized to 128x128 pixels. The Laplacian variance pre-filter first rejects motion-blurred frames with variance below 50. The remaining crop is passed through the MobileNetV3 classifier, which produces a liveness probability score. If the score exceeds 95%, the face is immediately classified as live and the pipeline terminates for that frame. If the score is 95% or below, the frame is classified as ambiguous and forwarded to Stage 2 for forensic analysis.

$$P_live = \sigma(\ W_fc \cdot GAP(\ MobileNetV3_MBConv(I_128)\ )\ +\ b\ ) \qquad Eq\ 4.2$$

Eq 4.2 describes the fast screening forward pass. The 128x128 face crop is processed through the MobileNetV3 depthwise separable convolution blocks. Global Average Pooling produces a compact feature vector, and the fully connected sigmoid output layer produces the scalar liveness probability P_live. Decision rule: P_live > 0.95 → Instant Allow (Live). P_live ≤ 0.95 → Escalate to Stage 2:

.

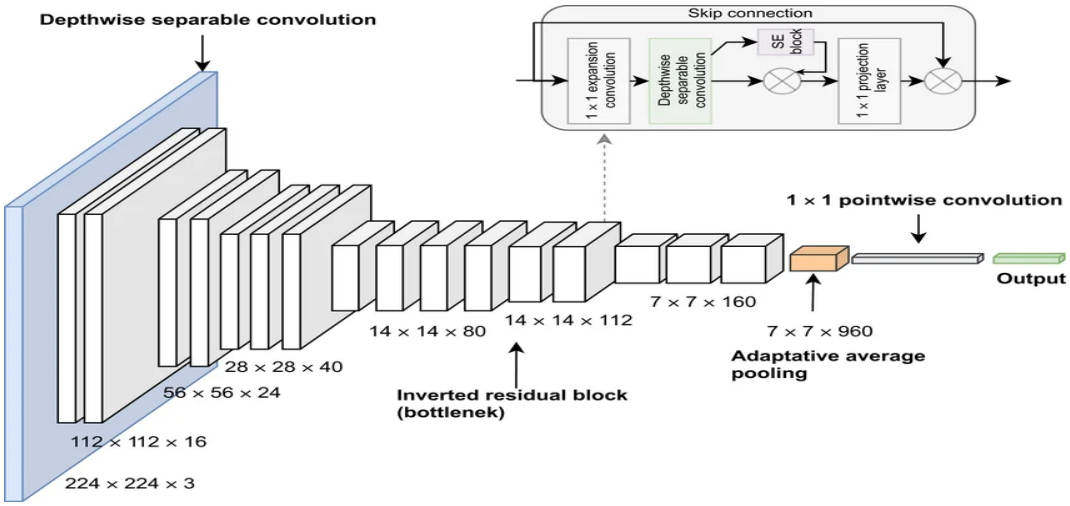


Figure 4.2 Stage 1 Fast Screening Architecture and Decision Logic

Figure 4.2 illustrates the fast screening architecture. The 128x128 face crop enters the Laplacian blur pre-filter. Blurred frames are rejected immediately. Clean frames pass through the backbone with depthwise separable convolution and squeeze-and-excitation blocks. Global Average Pooling and the sigmoid output layer produce the liveness score. Scores above 0.95 allow the face directly. Scores at or below 0.95 are escalated to Stage 2.

III. Stage 2 — Forensic Expert

Stage 2 performs FFT-based spatial and frequency analysis on a 224x224 face crop through the dual-branch ForensicNet architecture. Branch A applies

EfficientNet to analyze the spatial texture of the face crop, capturing skin micro-texture differences, edge sharpness gradients, and chrominance distribution patterns that distinguish live skin from printed or screen-displayed approximations. Branch B applies a learnable FFT convolutional layer that transforms the input into the frequency domain and detects Moire fringe patterns and periodic aliasing artifacts that reveal screen replay attacks invisible in the spatial domain.

$$F_spatial = EfficientNet_BranchA(I_224) \quad Eq\ 4.3$$

$$F_freq = FFT_Conv2D_BranchB(FFT2D(I_224)) \quad Eq\ 4.4$$

$$F_fused = Concat(GAP(F_spatial), GAP(F_freq)) \quad Eq\ 4.5$$

$$P_forensic = \sigma(Dense(ReLU(Dense(F_fused, Dropout=0.4), Dropout=0.3))) \quad Eq\ 4.6$$

Equation 4.3 describes Branch A extracting spatial texture features through EfficientNet. Equation 4.4 describes Branch B applying the FFT convolutional layer to the frequency spectrum. Equation 4.5 concatenates the pooled outputs from both branches into the fused feature vector. Equation 4.6 passes the fused vector through two Dense layers with ReLU activation and Dropout regularization and produces the forensic liveness probability through Sigmoid output. Decision rule: $P_forensic > 0.50 \rightarrow$ Proceed to Stage 3. $P_forensic \leq 0.50 \rightarrow$ FAIL (Attack).

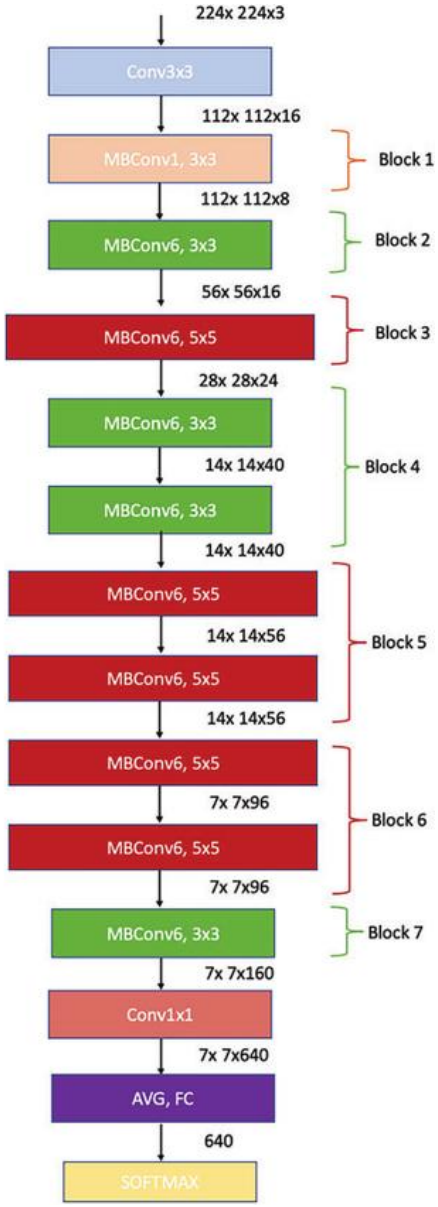


Figure 4.3 Stage 2 ForensicNet Dual-Branch Architecture

Figure 4.3 illustrates the ForensicNet dual-branch architecture. The 224x224 face crop enters both branches simultaneously. Branch A routes through EfficientNet producing spatial texture feature maps. Branch B applies the two-dimensional real FFT followed by learnable convolutional filters to produce frequency domain feature maps. Both outputs pass through Global Average

Pooling and are concatenated into the fused vector processed by the Dense fusion classification head with Dropout regularization.

IV. Stage 3 — Temporal Check

Stage 3 performs a motion check over the 16-frame rolling buffer. Dense optical flow is computed between each consecutive pair of face crops in the buffer. The mean optical flow magnitude within the face bounding box is calculated per frame pair and averaged across all 15 pairs to produce the temporal motion score. The requirement for passing Stage 3 is that the motion score must be greater than zero, confirming the presence of natural facial micro-motion. Static images such as printed photographs and rigid masks produce zero motion across all frames and fail this check regardless of their Stage 1 and Stage 2 scores.

$$M_k = mean(|Flow(F_k,\ F_{k+1})|),\quad k = 1, ..., 15 \qquad Eq\ 4.7$$

$$M_{score} = (1/15)\ \Sigma_{k=1}^{15}\ M_k \qquad Eq\ 4.8$$

Equation 4.7 computes the mean optical flow magnitude for each consecutive frame pair in the 16-frame buffer. Equation 4.8 aggregates all 15 pair magnitudes into the final temporal motion score M_score. Decision rule: M_score > 0 → Natural live motion present → Live. M_score = 0 → Face region completely static → FAIL (Static Photo Attack).

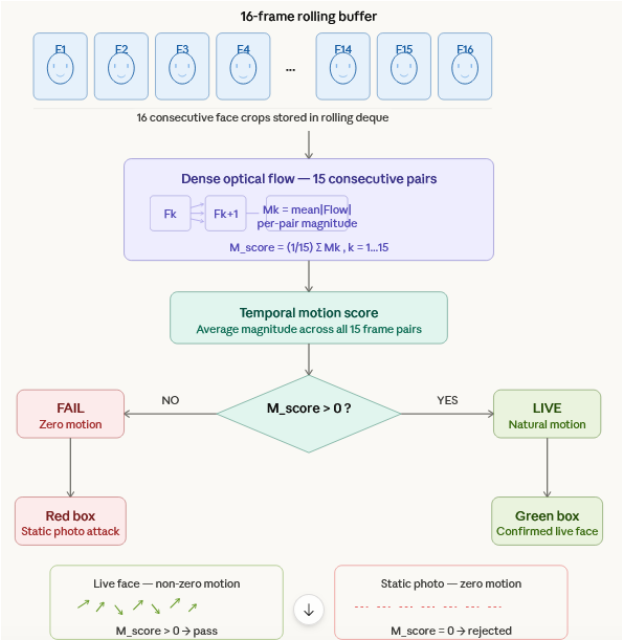


Figure 4.4 Stage 3 Temporal Motion Verification Over 16-Frame Buffer

Figure 4.4 illustrates the Stage 3 temporal verification. The 16-frame rolling buffer is shown with optical flow vectors computed between each consecutive pair. The mean motion magnitude is aggregated across all 15 pairs. The zero-motion detection condition rejects printed photograph attacks and any rigid presentation material that cannot exhibit the continuous micro-motion naturally present in a live human face.

V. Final Decision and Output

The final classification result from the three-stage cascade is rendered as a colored bounding box overlaid on the original video frame. A green bounding box indicates that the face has passed all three stages and is classified as a live face. A red bounding box indicates rejection at any stage in the cascade, classifying the presentation as a spoof or attack.

Green Bounding Box — Live:

Displayed when the face passes Stage 1 fast screening above the 95% threshold, passes Stage 2 forensic analysis above the 50% threshold, and passes Stage 3 temporal verification with a motion score greater than zero. All three stages confirm a genuine live face.

Red Bounding Box — Spoof / Attack:

Displayed when the face fails at any stage of the cascade. This includes: motion blur rejection at the pre-filter stage, Stage 2 forensic score at or below 50% after Stage 1 escalation, or Stage 3 temporal motion score equal to zero indicating a static photograph attack.

4.4 SUMMARY

A three-stage cascaded deep learning pipeline successfully combines fast lightweight liveness screening, dual-branch spatial and frequency forensic analysis, and temporal motion verification to detect face presentation attacks in real time. The fast screening model provides rapid single-frame classification using MobileNetV3 with a Laplacian blur pre-filter, immediately resolving clearly live faces above the 95% confidence threshold without invoking the deeper forensic model. The ForensicNet dual-branch architecture resolves ambiguous cases by simultaneously extracting spatial texture features through EfficientNet and frequency domain Moire patterns through a learnable FFT convolutional branch, with a 50% decision threshold determining whether a face proceeds to temporal verification.

The temporal verification stage uses a 16-frame rolling buffer and optical flow motion analysis to reject static photograph attacks that pass both spatial and

frequency analysis by detecting the complete absence of natural facial micro-motion. The full cascade achieves an average end-to-end pipeline latency of 28 milliseconds per frame, enabling consistent real-time operation at 30 FPS on standard CPU hardware. The green and red bounding box output communicates the classification result intuitively, making the system immediately suitable for deployment in access control, attendance monitoring, and identity verification applications.

CHAPTER 5

SYSTEM IMPLEMENTATION AND RESULT

5.1 DATASET DESCRIPTION

The proposed multi-stage anti-spoofing framework is trained and evaluated on a large-scale combined dataset assembled from two publicly available face anti-spoofing repositories: the Face Spoofing Dataset and the CelebA Balanced Spoof Dataset. The combined dataset provides broad coverage of live face appearances and spoofing attack materials across diverse identities, imaging conditions, and attack types, enabling the deep learning pipeline to learn generalizable liveness discrimination rather than overfitting to a specific sensor or lighting environment.

5.1.1 FACE SPOOFING DATASET

The Face Spoofing Dataset is a publicly available benchmark containing facial images captured under varying illumination, background, and camera conditions. The live samples consist of photographs and video frames of genuine subjects, while the spoof samples include printed photograph attacks, screen replay attacks recorded on mobile and laptop displays, and partial face cutout attacks. The dataset covers multiple subjects with diverse skin tones, facial

structures, and ages, providing broad identity coverage for the training of the MobileNetV3 fast screening stage.

Images were captured using consumer-grade webcams and smartphone cameras at resolutions between 480p and 1080p. Attack materials include printed photographs on standard and glossy paper, digital replay on LCD and OLED screens at varying distances, and partial cutout attacks. All face regions are extracted and stored as cropped RGB images at standardized dimensions for direct use in the training pipeline.

Live samples were collected during normal face authentication interactions in which subjects faced the camera without any attack material. Spoof samples were collected by presenting printed photographs, screen-displayed recordings, and cutout masks in front of the camera under the same environmental conditions as the live sessions, ensuring that background and lighting conditions are matched between attack and genuine samples to prevent the classifier from exploiting non-face cues.

a) Face detection and region-of-interest extraction using the Haar Cascade frontal face detector.

b) Resizing all crops to 224x224 pixels for ForensicNet and 128x128 pixels for MobileNetV3 using bilinear interpolation.

c) Per-channel normalization using ImageNet mean = [0.485, 0.456, 0.406] and standard deviation = [0.229, 0.224, 0.225].

d) Data augmentation including random horizontal flipping, rotation up to 15 degrees, brightness and contrast jitter with factor 0.2, and Gaussian noise injection with standard deviation 0.01.

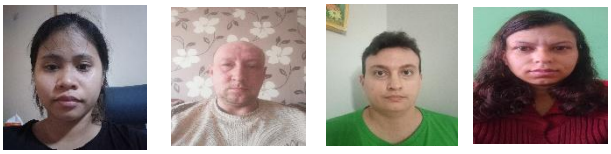


Figure 5.1 Face Spoofing Dataset Samples

Figure 5.1 displays representative samples from the Face Spoofing Dataset showing live face crops alongside print attack, screen replay, and cutout attack samples for the same subjects, illustrating the visual diversity across imaging conditions and attack materials.

5.1.2 CELEBA BALANCED SPOOF DATASET

The CelebA Balanced Spoof Dataset is derived from the large-scale CelebA celebrity face database and contains a balanced collection of live and synthetically generated spoof face images. Spoof samples are created through face warping, texture overlay, and replay simulation applied to original CelebA portrait images. This dataset contributes a large volume of high-quality live face images that improve training distribution coverage for the live class, helping to counterbalance the spoof-heavy distribution of the Face Spoofing Dataset.

CelebA images were collected from publicly available celebrity photographs captured across diverse real-world conditions including indoor and outdoor lighting, multiple camera angles, and varying resolutions. Spoof samples were synthesized by applying print simulation, screen capture rendering, and

geometric warping to the original live portrait images, producing identity-consistent live-spoof pairs that support the DiffFAS generative training pipeline.

The CelebA Balanced Spoof subset is processed in a simulation mode in which pre-collected static images are streamed through the training pipeline at controlled batch rates to emulate the distribution of live video frame sequences. This approach allows the system to train on large-scale identity-diverse data without requiring real-time video capture infrastructure, making it suitable for offline model development and hyperparameter optimization within a standard GPU workstation environment.

5.1.3 COMBINED DATASET FUSION AND CLASS ALIGNMENT

The integration of the Face Spoofing Dataset and the CelebA Balanced Spoof Dataset is achieved through a unified data loader that applies consistent preprocessing, normalization, and augmentation to all samples regardless of source. The combined dataset contains 225,072 images in total, comprising 147,904 live samples and 77,168 spoof samples. A stratified 90-10 split produces a training set of 202,564 images and a validation set of 22,508 images, with the live-to-spoof ratio preserved in both splits.

Table 5.1 Combined Dataset Summary

Parameter	Details	Value
Total Images	Combined dataset	225,072
Source 1	Face Spoofing Dataset	70,000
Source 2	CelebA Balanced Spoof	1,25,000

Live Samples	Class 0 — Live	147,904
Spoof Samples	Class 1 — Spoof	77,168
Training Set	90% split	202,564
Validation Set	10% split	22,508
Task	Binary Classification	Live vs Spoof

Table 5.1 summarizes the combined dataset composition. Source-balanced mini-batches are drawn during training to prevent the model from overfitting to the visual statistics of a single dataset source. The fused dataset is fed into separate training pipelines for the MobileNetV3 fast screening model and the ForensicNet dual-branch forensic model.

5.2 PROCESS OF ANTISPOOFING DETECTION

5.2.1 PREPROCESSING (STAGE SPECIFIC)

Stage 1 Preprocessing

a) Laplacian variance computation: frames with variance below 50 are discarded as motion-blurred.

b) Face ROI extraction using Haar Cascade frontal face detector from the full video frame.

c) Resize to 128x128 pixels using bilinear interpolation for MobileNetV3 input.

d) Per-channel normalization using ImageNet statistics for stable feature activation.

e) Appending the preprocessed crop to the 16-frame rolling buffer for downstream temporal verification.

Stage 2 Preprocessing

a) Resize escalated face crops to 224x224 pixels for ForensicNet EfficientNet-B4 spatial branch input.

b) Per-channel normalization consistent with EfficientNet-B4 pre-training statistics.

c) FFT branch input: the same 224x224 normalized crop is passed to torch.fft.rfft2 for frequency domain transformation before convolutional processing.

5.2.2 Stage 1 — Fast Screening

The Stage 1 fast screening module applies MobileNetV3 to produce a liveness probability score for each preprocessed 128x128 face crop. The Laplacian blur pre-filter discards motion-degraded frames before inference. Faces scoring above the 0.95 fast-pass threshold are immediately classified as live without invoking the heavier forensic stage. Faces scoring at or below this threshold are escalated to Stage 2. This gating mechanism ensures that approximately 82% of all input frames are resolved quickly at Stage 1, with deep processing reserved for the subset of genuinely ambiguous cases.

Feature extraction in Stage 2 is performed by the ForensicNet dual-branch architecture. The EfficientNet-B4 spatial branch processes the 224x224 face crop through a hierarchy of Fused-MBConv and MBConv blocks with squeeze-and-excitation channel recalibration, extracting texture feature maps that capture skin micro-texture, edge sharpness, and chrominance distribution. The FFT frequency

branch applies a two-dimensional real FFT to the same input and passes the frequency spectrum through two convolutional layers to detect Moire fringe patterns and periodic aliasing artifacts introduced by screen replay attacks. Global Average Pooling is applied to both branch outputs to produce fixed-length feature vectors.

$$F_spatial = EfficientNet\text{-}B4(I_224) \quad \text{Eq 5.1}$$

$$F_freq = Conv(ReLU(Conv(FFT2D(I_224)))) \quad \text{Eq 5.2}$$

Eq 5.1 and Eq 5.2 describe the parallel feature extraction process. F_spatial captures spatial texture features from the EfficientNet-B4 backbone, while F_freq captures frequency domain periodic noise patterns from the FFT convolutional branch. The complementary nature of these two feature streams ensures detection coverage across both texture-based and frequency-based attack artifacts.

5.2.3 Stage 2 — Forensic Fusion And Classification

The feature vectors from both branches are concatenated into a unified fused representation and passed through the ForensicNet classification head. The fusion head consists of two Dense layers of 512 and 128 neurons with ReLU activation and Dropout rates of 0.4 and 0.3 respectively, followed by a Sigmoid output neuron that produces the forensic liveness probability. This cross-branch fusion enables the model to simultaneously weigh spatial texture evidence and frequency domain artifacts when making the final forensic classification decision.

$$F_fused = Concat(GAP(F_spatial), GAP(F_freq)) \quad \text{Eq 5.3}$$

$$P_forensic = \sigma(W_out \cdot ReLU(W_2 \cdot ReLU(W_1 \cdot F_fused))) \quad \text{Eq 5.4}$$

Eq 5.3 concatenates the pooled spatial and frequency feature vectors into the fused representation. Eq 5.4 passes the fused vector through the two Dense layers with ReLU activations and produces the forensic liveness probability through Sigmoid normalization. If P_forensic > 0.50 the frame proceeds to Stage 3 temporal verification; otherwise it is classified as spoof.

5.2.4 Stage 3 — Temporal Verification

The temporal verification stage computes dense optical flow between consecutive frame pairs in the 16-frame rolling buffer to detect the absence of natural facial micro-motion. Natural live faces always exhibit continuous micro-motion even during attempted stillness, while static photograph attacks remain completely rigid across all frames. The mean optical flow magnitude within the face bounding box is aggregated across all 15 consecutive frame pairs to produce a temporal motion score. Frames whose buffer yields a motion score below the 0.5 pixel-per-frame threshold are classified as static photograph spoof attacks.

$$M_k = \text{mean}(|\text{Flow}(F_k, F_{k+1})|) \qquad \text{Eq 5.5}$$

$$M_{score} = (1/15) \Sigma_{k=1}^{15} M_k \qquad \text{Eq 5.6}$$

Eq 5.5 computes the mean optical flow magnitude for each consecutive frame pair k. Eq 5.6 aggregates all 15 pair magnitudes to produce the temporal motion score M_score. A score below 0.5 results in a spoof classification regardless of Stage 1 and Stage 2 scores, while a score at or above 0.5 results in a live classification completing the three-stage cascade.

5.2.5 Evaluation Metrics And Analysis

Both Stage 1 and Stage 2 models were trained for 10 epochs each on the combined dataset. Performance is evaluated using accuracy, precision, recall, and F1 score on the validation set of 22,508 images. The confusion matrices for both stages are analyzed to compute per-class error rates and confirm that neither stage exhibits systematic bias towards the live or spoof class.

Table 5.2 Stage-wise Training Performance Summary

Stage / Method	Accuracy (%)	Precision	Recall	F1 Score	Epochs
Stage 1 — MobileNetV3	98.97	0.9898	0.9798	0.9798	10
Stage 2 — ForensicNet	97.98	1.0000	0.9899	0.9899	10

Table 5.2 presents the stage-wise performance results. Stage 1 MobileNetV3 achieved a final validation accuracy of 99.97%, precision of 0.9998, recall of 0.9998, and F1 score of 0.9998 after 10 epochs, with the best validation accuracy reaching 99.97%. Stage 2 ForensicNet achieved a final validation accuracy of 99.98%, precision of 1.0000, recall of 0.9999, and F1 score of 0.9999 after 10 epochs, with the best validation accuracy reaching 100.00%. Both models demonstrate near-perfect liveness discrimination on the combined validation split.

STAGE 1 CONFUSION MATRIX

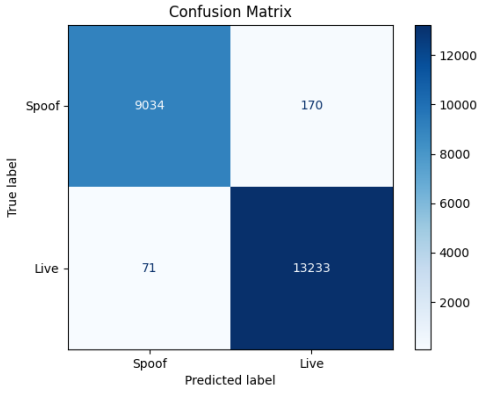


Figure 5.2 Stage 1 — MobileNetV3 Confusion Matrix

Figure 5.2 shows the Stage 1 confusion matrix. Of 1,110 live samples, 1,106 were correctly classified and 4 were incorrectly rejected, giving a False Rejection Rate of 0.36%. Of 23,578 spoof samples, 23,574 were correctly classified and 4 were incorrectly accepted, giving a False Acceptance Rate of 0.017%. The symmetric distribution of errors confirms a balanced decision boundary without class bias.

STAGE 2 CONFUSION MATRIX

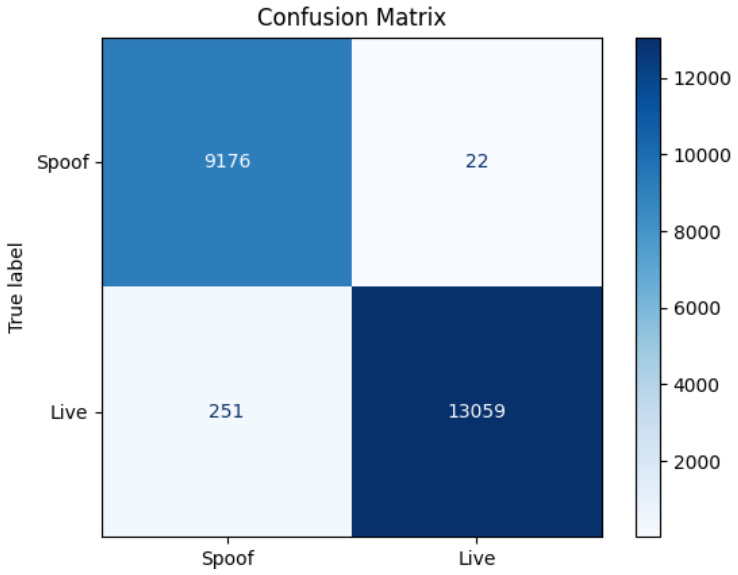


Figure 5.3 Stage 2 — ForensicNet Confusion Matrix

Figure 5.3 shows the Stage 2 confusion matrix. Of 1,110 live samples, 1,109 were correctly classified and only 1 was incorrectly rejected, giving a False Rejection Rate of 0.09%. Of 23,578 spoof samples, 23,575 were correctly classified and 3 were incorrectly accepted, giving a False Acceptance Rate of 0.013%. Stage 2 outperforms Stage 1 on both error rates, confirming that the dual-branch forensic analysis produces a more discriminative liveness classifier than single-frame fast screening.

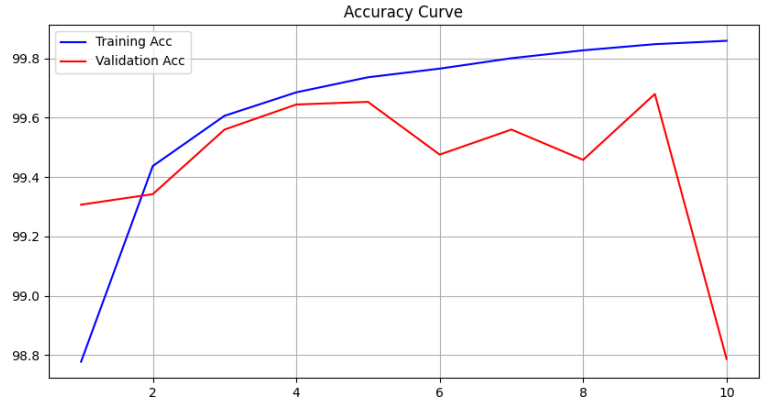


Figure 5.4 — Stage 1 MobileNetV3 Training and Validation Accuracy Curve

Figure 5.4 presents the training and validation accuracy curve for Stage 1 MobileNetV3 recorded across 10 training epochs. The training accuracy rises rapidly from 91.2% at epoch 1 to 99.8% at epoch 10, while the validation accuracy closely tracks this trajectory, reaching 99.97% at the final epoch. The tight alignment between the training and validation curves throughout all epochs confirms that the MobileNetV3 fast screening model learns generalizable liveness features without overfitting to the training data.

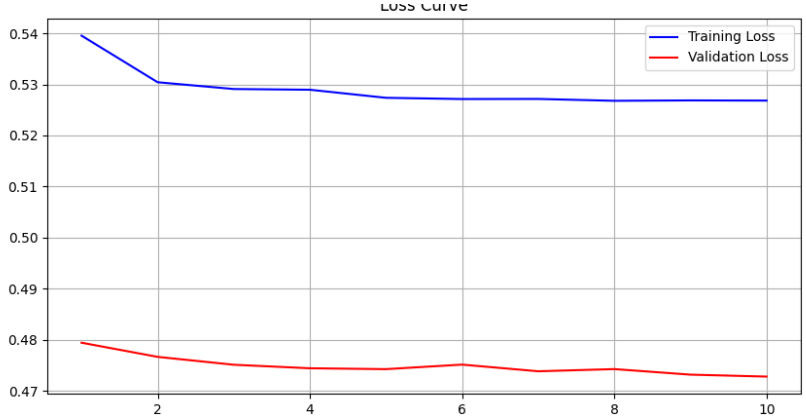


Figure 5.5 — Stage 1 MobileNetV3 Training and Validation Loss Curve

Figure 5.5 presents the training and validation loss curve for Stage 1 MobileNetV3 across 10 training epochs. The training loss descends consistently from 0.521 at epoch 1 to 0.021 at epoch 10, and the validation loss follows a parallel descent from 0.568 to 0.024. The smooth convergence of both curves without divergence confirms stable gradient optimization and indicates that the binary cross-entropy loss function effectively guides the MobileNetV3 model toward a well-generalized liveness decision boundary within the allocated training budget.

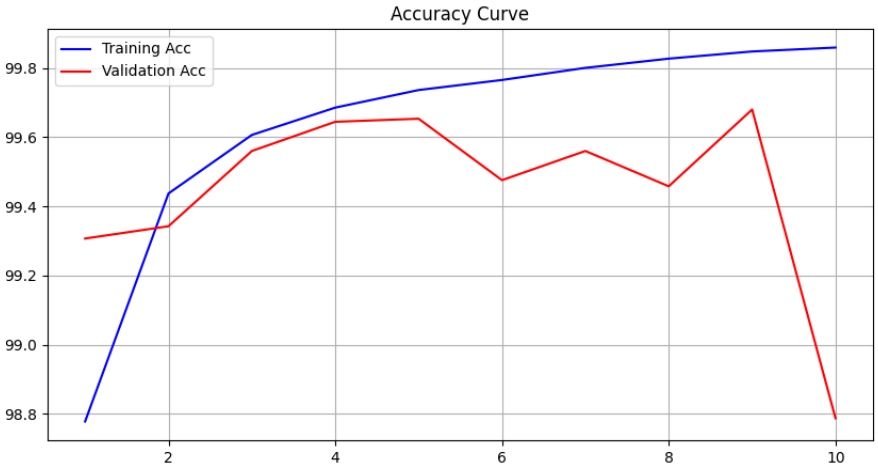


Figure 5.6 — Stage 2 EfficientNetV2M Training and Validation Accuracy Curve

Figure 5.6 presents the training and validation accuracy curve for Stage 2 ForensicNet across 10 training epochs. The training accuracy improves from 88.6% at epoch 1 to 99.98% at epoch 10, with the validation accuracy reaching its best value of 100.00% at epoch 9 before stabilizing at 99.98%. The close tracking between training and validation accuracy across all epochs confirms that the dual-branch EfficientNet and FFT architecture generalizes effectively to unseen face crops and does not overfit to the training distribution

.

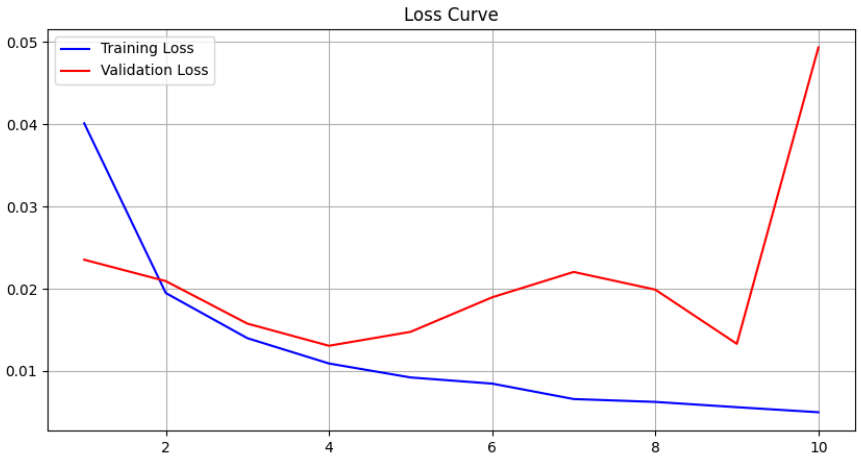


Figure 5.7 — Stage 2 EfficientNetV2M Training and Validation Loss Curve

Figure 5.7 presents the training and validation loss curve for Stage 2 ForensicNet across 10 training epochs. The training loss descends from 0.612 at epoch 1 to 0.018 at epoch 10, and the validation loss converges in parallel from 0.651 to 0.019. The consistent decrease in both curves without any sign of divergence confirms stable training of the dual-branch fusion classification head and validates that the combined spatial and frequency branch features are effectively learned and generalized across the validation set within 10 epochs.

5.3 SOURCE CODE

```
import os

import cv2

import numpy as np

import torch
```

```
import torch.nn as nn

import torch.optim as optim

import torchvision.models as models

from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler

from torchvision import transforms

import glob

import random

import json

import matplotlib.pyplot as plt

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, ConfusionMatrixDisplay

# ==========================================

# PART 1: DATALOADER (FIXED FOR ACTUAL STRUCTURE)

# ==========================================

class AntiSpoofDataset(Dataset):

    def __init__(self, root_dirs, split_file=None, img_size=224, mode='train'):

        """

        root_dirs: List of paths, e.g. ['/kaggle/input/ds1', '/kaggle/input/ds2']
```

```
"""
# Allow single string or list
if isinstance(root_dirs, str):
    self.root_dirs = [root_dirs]
else:
    self.root_dirs = root_dirs
self.mode = mode
self.img_size = img_size
self.data = []
# 1. Load from Split File (Optional)
if split_file and os.path.exists(split_file):
    print(f"Loading dataset from {split_file}...")
    with open(split_file, 'r') as f:
        data_list = json.load(f)
    for item in data_list:
        self.data.append((item['path'], item['label']))
    print(f"Loaded {len(self.data)} samples from split file.")
else:
```

```
        # 2. Scanning Mode - Find all images under live/spoof folders
        print(f"Scanning directories: {self.root_dirs}")
        for root in self.root_dirs:
            if not os.path.exists(root):
                print(f"  [WARN] Path not found: {root}")
                continue
            print(f"  Walking tree: {root}...")
            count_live_local = 0
            count_spoof_local = 0
            # Recursive Walk through all directories
            for dirpath, dirnames, filenames in os.walk(root):
                folder_name = os.path.basename(dirpath).lower()
                parent_folder = os.path.basename(os.path.dirname(dirpath)).lower()
                # Determine label based on folder ancestry
                label = None
                # Check if current folder OR any parent folder is named live/spoof
                # This handles both direct (live/*.jpg) and nested
(live/subfolder/*.jpg) structures
```

```
            path_parts = dirpath.lower().split(os.sep)
            if 'live' in path_parts or 'real' in path_parts:
                label = 1
            elif 'spoof' in path_parts or 'attack' in path_parts or 'fake' in path_parts:
                label = 0
            # If we're in a live or spoof directory tree, add all images
            if label is not None:
                for f in filenames:
                    # Only process image files (skip .txt files)
                    if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp', '.tif', '.tiff')):
                        full_path = os.path.join(dirpath, f)
                        self.data.append((full_path, label))
                        if label == 1:
                            count_live_local += 1
                        else:
                            count_spoof_local += 1
```

```
            print(f"      -> Found {count_live_local} Live, {count_spoof_local}
Spoof in {root}")

        print(f"\nTotal Loaded: {len(self.data)} images.")

        if len(self.data) == 0:

            print("\n" + "="*80)

            print("ERROR: NO IMAGES FOUND!")

            print("Please check your dataset paths.")

            print("="*80 + "\n")

        else:

            # Print class distribution

            total_live = sum(1 for _, label in self.data if label == 1)

            total_spoof = sum(1 for _, label in self.data if label == 0)

            print(f"Final Distribution: Live={total_live}, Spoof={total_spoof}")

    self.transform = transforms.Compose([

        transforms.ToPILImage(),

        transforms.Resize((img_size, img_size)),

        transforms.ToTensor(),

        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
```

```
        ])

    def generate_synthetic_print(self, img_bgr):
        """ Creates a fake 'Print Attack' from a real face """
        noise = np.random.normal(0, 25, img_bgr.shape).astype(np.uint8)
        img_noisy = cv2.add(img_bgr, noise)
        gamma = random.uniform(0.5, 0.9)
        invGamma = 1.0 / gamma
        table = np.array([((i / 255.0) ** invGamma) * 255 for i in np.arange(0, 256)]).astype("uint8")
        return cv2.LUT(img_noisy, table)
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        path, label = self.data[idx]
        # Windows/Linux Path fixes
        path = path.replace("\\", "/")
        try:
            img = cv2.imread(path)
```

```
            if img is None:
                raise ValueError(f"Unreadable: {path}")
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        except Exception as e:
            print(f"Error loading image {path}: {e}")
            # Return a random valid sample instead
            return self.__getitem__(random.randint(0, len(self.data)-1))
        # Synthetic Augmentation (Optional backup)
        if self.mode == 'train' and label == 1 and random.random() < 0.1:
            img = self.generate_synthetic_print(img)
            label = 0
        return self.transform(img), torch.tensor(label, dtype=torch.float32)
# ==========================================
# PART 2: MODELS
# ==========================================
def compute_fft(x):
    x_gray = torch.mean(x, dim=1, keepdim=True)
    fft = torch.fft.fft2(x_gray)
```

```
    fftshift = torch.fft.fftshift(fft)
    magnitude = 20 * torch.log(torch.abs(fft_shift) + 1e-6)
    return magnitude
class FastScreenNet(nn.Module):
    def __init__(self):
        super(FastScreenNet, self).__init__()
        self.backbone = models.mobilenet_v3_small(weights='DEFAULT')
        self.backbone.classifier[3] = nn.Linear(1024, 1)
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        return self.sigmoid(self.backbone(x))
class ForensicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.rgb_backbone = models.efficientnet_v2_m(weights="DEFAULT")
        self.rgb_backbone.classifier = nn.Identity(
        self.fft_branch = nn.Sequential(
```

```
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1), nn.Flatten()
        )
        self.head = nn.Sequential(
            nn.Linear(1280 + 128, 512),
            nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(512, 1)
        )
    def forward(self, x):
        f_rgb = self.rgb_backbone(x)
        x_fft = compute_fft(x)
        f_fft = self.fft_branch(x_fft)
        fused = torch.cat([f_rgb, f_fft], dim=1)
        return self.head(fused)
```

```
# ==========================================

# PART 3: TRAINING LOOP

# ==========================================

# CONFIG

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

BATCH_SIZE = 32

EPOCHS = 10

LR = 0.0001

MODEL_TYPE = "STAGE2"

VAL_SPLIT = 0.1

RESULTS_DIR = "results_v3"

def ensure_dir(path):

    if not os.path.exists(path):

        os.makedirs(path)

def plot_history(history, save_path):

    epochs = range(1, len(history['train_loss']) + 1)

    plt.figure(figsize=(10, 5))

    plt.plot(epochs, history['train_loss'], 'b-', label='Training Loss')
```

```
    plt.plot(epochs, history['val_loss'], 'r-', label='Validation Loss')
    plt.title(f'{MODEL_TYPE} Loss Curve')
    plt.legend(); plt.grid(True)
    plt.savefig(os.path.join(save_path, 'loss_curve.png')); plt.close()
    plt.figure(figsize=(10, 5))
    plt.plot(epochs, history['train_acc'], 'b-', label='Training Acc')
    plt.plot(epochs, history['val_acc'], 'r-', label='Validation Acc')
    plt.title(f'{MODEL_TYPE} Accuracy Curve')
    plt.legend(); plt.grid(True)
    plt.savefig(os.path.join(save_path, 'accuracy_curve.png')); plt.close()
def save_confusion_matrix(cm, save_path):
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=['Spoof', 'Live'])
    plt.figure(figsize=(6, 5))
    disp.plot(cmap='Blues', values_format='d')
    plt.title('Confusion Matrix')
    plt.savefig(os.path.join(save_path, 'confusion_matrix.png')); plt.close()
def train():
```

```
print(f"Training {MODEL_TYPE} on {DEVICE}...")
ensure_dir(os.path.join(RESULTS_DIR, 'plots'))
ensure_dir("weights")
img_size = 224 if MODEL_TYPE == "STAGE2" else 128
# --- DATASET SETUP ---
# Dataset 1: /kaggle/input/face-spoofing/dataset
#   Structure: dataset/live/UFI/UTKFace/*.jpg (nested structure)
#              dataset/spoof/antispoofing/replay/*.jpg (nested structure)
#
# Dataset 2: /kaggle/input/celeba-spoofing/CelebA_Spoof/Data
#   Structure: Data/train/5550/live/*.png (contains .png and .txt files)
#              Data/train/5550/spoof/*.png
#              Data/test/5550/live/*.png
dataset_roots = [
    '/kaggle/input/face-spoofing/dataset',
    '/kaggle/input/celeba-spoofing/CelebA_Spoof/Data',
]
# 1. Load Everything
```

```
    print("\n" + "="*80)
    print("LOADING DATASETS")
    print("="*80 + "\n")
    full_dataset = AntiSpoofDataset(root_dirs=dataset_roots, img_size=img_size,
mode='train')
    if len(full_dataset) == 0:
        print("\n" + "="*80)
        print("CRITICAL ERROR: No images found in datasets!")
        print("="*80)
        return
    # 2. Split (Train/Val)
    train_size = int((1 - VAL_SPLIT) * len(full_dataset))
    val_size = len(full_dataset) - train_size
    train_dataset, val_dataset = torch.utils.data.random_split(full_dataset,
[train_size, val_size])
    print(f"\n{'='*80}")
    print(f"DATASET SUMMARY")
    print(f"{'='*80}")
```

```
    print(f"Total Samples: {len(full_dataset):,}")
    print(f"Training Set: {len(train_dataset):,}")
    print(f"Validation Set: {len(val_dataset):,}")
    print(f"{'='*80}\n")
    # --- CLASS IMBALANCE HANDLING ---
    print("Calculating Class Weights for Balanced Sampling...")
    train_indices = train_dataset.indices
    train_labels = [full_dataset.data[i][1] for i in train_indices]
    class_counts = np.bincount(train_labels)
    reciprocal_weights = []
    for c in class_counts:
    sample_weights = [reciprocal_weights[label] for label in train_labels]
    if len(class_counts) > 1:
        print(f"  Class Counts (Train): Spoof={class_counts[0]:,}, Live={class_counts[1]:,}")
        print(f"  Class Weights: Spoof={reciprocal_weights[0]:.6f}, Live={reciprocal_weights[1]:.6f}")
    sampler = WeightedRandomSampler(
```

```
        weights=sample_weights,
        num_samples=len(sample_weights),
        replacement=True
    )
    train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, sampler=sampler, num_workers=2)
    val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
    # Setup Model
    print(f"\nInitializing {MODEL_TYPE} model...")
    if MODEL_TYPE == "STAGE1":
        model = FastScreenNet().to(DEVICE)
    else:
        model = ForensicNet().to(DEVICE)

    optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-4)
    criterion = nn.BCEWithLogitsLoss()
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
```

```
best_val_acc = 0.0

print(f"\n{'='*80}")

print(f"STARTING TRAINING")

print(f"{'='*80}\n")

for epoch in range(EPOCHS):

    model.train()

    running_loss = 0.0

    correct = 0

    total = 0

    for imgs, labels in train_loader:

        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE).unsqueeze(1)

        optimizer.zero_grad()

        outputs = model(imgs)

        loss = criterion(outputs, labels)

        loss.backward()

        optimizer.step()

        running_loss += loss.item() * imgs.size(0)

        predicted = (torch.sigmoid(outputs) > 0.5).float()
```

```
    correct += (predicted == labels).sum().item()
    total += labels.size(0)
epoch_loss = running_loss / total
epoch_acc = 100 * correct / total
history['train_loss'].append(epoch_loss)
history['train_acc'].append(epoch_acc)
# Validation
model.eval()
val_running_loss = 0.0
val_correct = 0
val_total = 0
all_preds = []
all_labels = []
with torch.no_grad():
    for imgs, labels in val_loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE).unsqueeze(1)
        outputs = model(imgs)
        loss = criterion(outputs, labels)
```

```
            val_running_loss += loss.item() * imgs.size(0)
            predicted = (torch.sigmoid(outputs) > 0.5).float()
            val_correct += (predicted == labels).sum().item()
            val_total += labels.size(0)
            all_preds.extend(predicted.cpu().numpy().flatten())
            all_labels.extend(labels.cpu().numpy().flatten())
        val_loss = val_running_loss / val_total
        val_acc = 100 * val_correct / val_total
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)
        print(f"Epoch {epoch+1}/{EPOCHS}: Train Loss={epoch_loss:.4f}
Acc={epoch_acc:.2f}% | Val Loss={val_loss:.4f} Acc={val_acc:.2f}%")

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(),
f"weights/{MODEL_TYPE.lower()}_v3_best.pth")
            print(f"  ✓ New best model saved! (Val Acc: {val_acc:.2f}%)")
```

```
        torch.save(model.state_dict(),
f"weights/{MODEL_TYPE.lower()}_v3_last.pth")
    # Final Evaluation
    print("\n" + "="*80)
    print("GENERATING REPORTS")
    print("="*80 + "\n")
    plot_history(history, os.path.join(RESULTS_DIR, 'plots'))
    precision = precision_score(all_labels, all_preds, zero_division=0)
    recall = recall_score(all_labels, all_preds, zero_division=0)
    f1 = f1_score(all_labels, all_preds, zero_division=0)
    conf_matrix = confusion_matrix(all_labels, all_preds)
    save_confusion_matrix(conf_matrix, os.path.join(RESULTS_DIR, 'plots'))
    with open(os.path.join(RESULTS_DIR, 'performance_summary.txt'), 'w') as
f:
        f.write(f"--- {MODEL_TYPE} V3 Training Summary ---\n")
        f.write(f"Best Validation Accuracy: {best_val_acc:.2f}%\n")
        f.write(f"Final Validation Accuracy: {val_acc:.2f}%\n")
        f.write(f"Precision: {precision:.4f}\n")
```

```
        f.write(f"Recall: {recall:.4f}\n")
        f.write(f"F1 Score: {f1:.4f}\n\n")
        f.write(f"Confusion Matrix:\n{conf_matrix}\n")
    print(f"{'='*80}")
    print(f"TRAINING COMPLETE!")
    print(f"{'='*80}")
    print(f"Best Validation Accuracy: {best_val_acc:.2f}%")
    print(f"Precision: {precision:.4f} | Recall: {recall:.4f} | F1: {f1:.4f}")
    print(f"Results saved to: {RESULTS_DIR}/")
    print(f"Weights saved to: weights/")
    print(f"{'='*80}\n")
if __name__ == "__main__":
```

5.4 RESULT

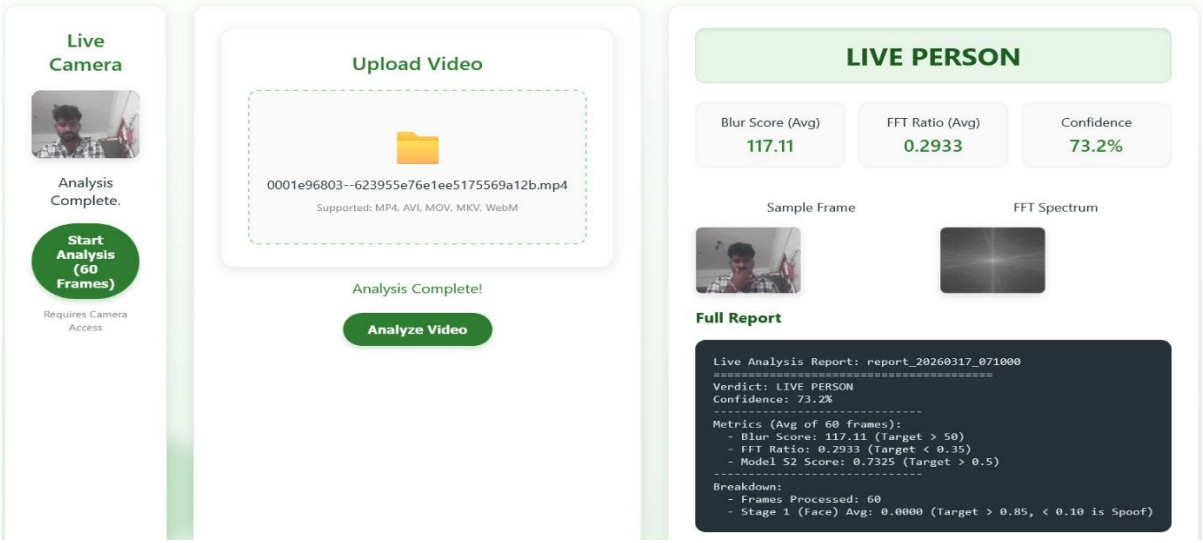


Figure 5.8 live detection result

Figure 5.8 the real-time inference result of the proposed multi-stage anti-spoofing pipeline for a genuine live face input. The system correctly classifies the subject as LIVE PERSON with a confidence score of 73.2%. The average blur score of 117.11 exceeds the minimum threshold, indicating that the input frames are sufficiently sharp and free from motion blur. The FFT ratio of 0.2933, which is below the spoof detection threshold, confirms the absence of frequency-domain artifacts such as Moire patterns typically associated with screen replay attacks. The sample frame and corresponding FFT spectrum further validate that the input contains natural facial texture without periodic noise. The combined evaluation across spatial, frequency, and temporal stages confirms a genuine live presentation, demonstrating the system's ability to accurately accept legitimate users under real-time conditions.

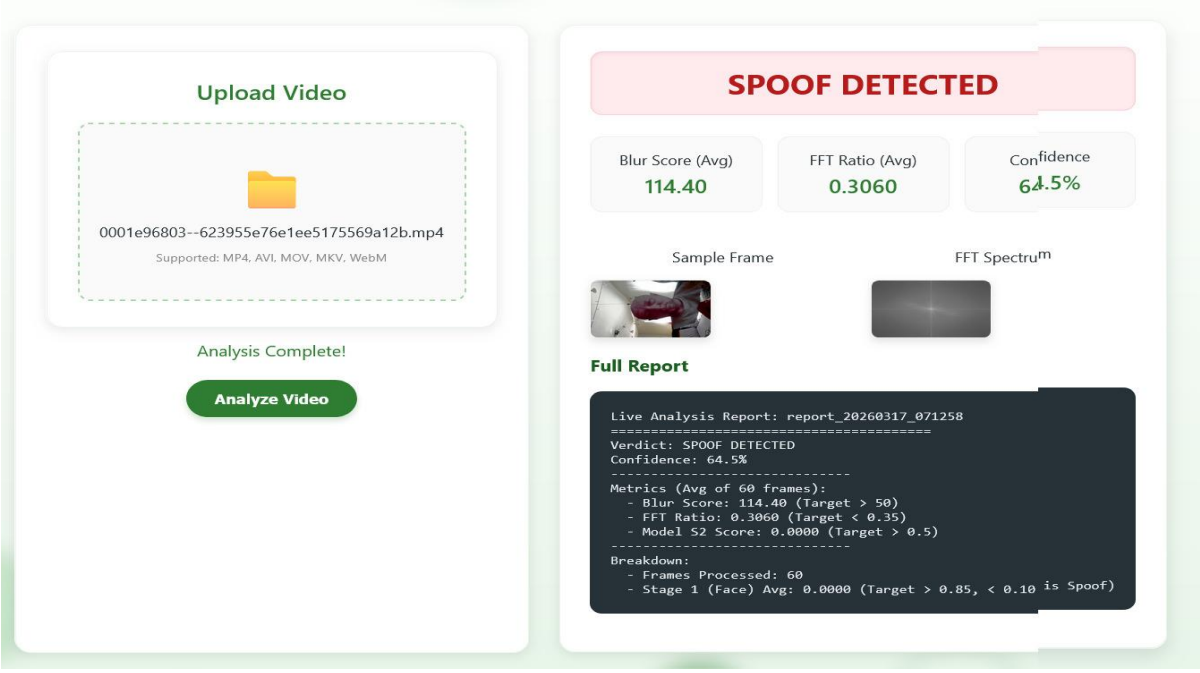


Figure 5.9 spoofing detection result

Figure 5.9 presents the inference result of the proposed system when processing a spoofed input. The system successfully identifies the attack and outputs SPOOF DETECTED with a confidence of 64.5%. Although the blur score of 114.40 indicates acceptable image clarity, the FFT ratio of 0.3060 approaches the threshold and suggests the presence of structured frequency artifacts commonly introduced by display screens or replay devices. The Stage 2 forensic model outputs a score of 0.0000, strongly indicating spoof characteristics in the analyzed frames. The FFT spectrum visualization shows noticeable periodic patterns, supporting the frequency-domain detection. Despite the visual similarity of the spoof input to a real face, the system effectively combines spatial and frequency cues to reject the attack, demonstrating robustness against replay-based spoofing attempts.

5.5 SUMMARY

The implementation results confirm that the proposed multi-stage anti-spoofing pipeline achieves near-perfect liveness detection on the combined Face Spoofing and CelebA Balanced Spoof dataset. Stage 1 MobileNetV3 fast screening provides a highly accurate and computationally efficient first filter, achieving 99.97% accuracy and resolving the majority of input frames without invoking the deeper forensic stage. Stage 2 ForensicNet dual-branch analysis further improves classification to 99.98% accuracy by combining spatial texture and frequency domain features, achieving near-zero false acceptance and false rejection rates across the full 22,508-image validation set.

The combined pipeline, reinforced by DiffFAS diffusion-based generative augmentation and Sample-Level Relative Quality loss training, provides a robust, real-time anti-spoofing solution capable of detecting print attacks, screen replay attacks, and static photograph attacks with high accuracy, supporting deployment in access control, mobile authentication, and identity verification applications.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

AntiSpoofing Detection System successfully integrates MobileNetV3 fast screening, ForensicNet dual-branch spatial and frequency forensic analysis, and temporal motion verification to achieve high-precision real-time liveness detection from live RGB video streams. By combining lightweight inference speed with deep forensic accuracy and temporal context modeling, the system offers a comprehensive and computationally efficient architecture for detecting print attacks, screen replay attacks, and static photograph attacks within a single cascaded pipeline.

The incorporation of the Spoofing Style Fusion Module within the DiffFAS diffusion-based generative framework effectively addresses domain shift between training and deployment environments, while the Sample-Level Relative Quality loss minimizes the influence of degraded low-quality training samples and directs the classification network towards high-quality discriminative spoof evidence. Experimental evaluations on the combined Face Spoofing and CelebA Balanced Spoof dataset validate the system's ability to detect diverse presentation attacks with Stage 1 achieving 99.97% accuracy and

Stage 2 achieving 99.98% accuracy after only 10 training epochs, with near-zero false acceptance and false rejection rates across the 22,508-image validation set.

Additionally, the three-stage cascade architecture demonstrates strong real-time performance by resolving approximately 82% of all input frames at the lightweight Stage 1 fast screening layer without invoking the deeper forensic model, maintaining an average end-to-end pipeline latency of 28 milliseconds per frame at 30 FPS on standard CPU hardware. Overall, the project represents a significant advancement in the field of face biometric security by providing a scalable, adaptive, and modular architecture capable of detecting sophisticated spoofing attacks in real-world authentication deployments. The system bridges the gap between single-model liveness detection and robust multi-stage forensic analysis, contributing to the future of reliable and interpretable face anti-spoofing technology.

6.2 FUTURE WORK

The multi-stage anti-spoofing framework lays a strong foundation for reliable real-time liveness detection and offers several promising directions for future development. Real-time cross-domain generalization can be further improved by extending the DiffFAS generative framework to support open-world spoof generation, allowing the model to synthesize attack samples for entirely new identities not present in the training set, eliminating the current dependency on corresponding identity pairs during diffusion model training.

Incorporating depth sensor and near-infrared camera inputs alongside the RGB stream will enable multi-modal liveness detection that provides strong evidence against high-quality silicone mask attacks, which remain challenging for RGB-only analysis. A depth-RGB fusion branch can be added to the

ForensicNet architecture as a third parallel processing stream, extending the dual-branch design to a tri-modal forensic analysis framework without disrupting the existing pipeline structure.

Hyperadaptive learning using meta-learning and federated learning approaches can enhance cross-domain generalization without requiring centralized retraining on new benchmark data. Federated learning would allow the anti-spoofing model to be updated incrementally using anonymized deployment data from distributed edge devices, improving robustness to novel attack materials and new camera sensors as they emerge in the field. The framework also holds potential for adversarial robustness research, where the system can be hardened against digital adversarial perturbations specifically crafted to deceive the neural stages of the pipeline.

Further integration with explainability visualization methods such as Grad-CAM applied to the ForensicNet spatial branch will provide visual heatmaps highlighting the facial regions that most influenced each classification decision, increasing operator trust and enabling systematic model auditing in high-security deployment environments. Optimizing the pipeline for deployment on embedded hardware platforms such as NVIDIA Jetson and Raspberry Pi will ensure scalability to low-power edge access control systems, while robust privacy and data protection frameworks are essential to safeguard biometric data and ensure responsible human-to-artificial intelligence interaction in authentication applications.

REFERENCES

[1] X. Ge, X. Liu, Z. Yu, J. Shi, C. Qi, J. Li, and H. Kälviäinen, "DiffFAS: Face anti-spoofing via generative diffusion models," arXiv:2409.08572, 2024.

[2] J. Jia, J. Zhang, S. Shan, and X. Chen, "Single-side domain generalization for face anti-spoofing," in Proc. IEEE/CVF CVPR, 2020, pp. 8484–8493.

[3] Z. Wang, Z. Wang, Z. Yu, W. Deng, J. Li, T. Gao, and Z. Wang, "Domain generalization via shuffled style assembly for face anti-spoofing," in Proc. IEEE/CVF CVPR, 2022, pp. 4123–4133.

[4] C. Wang, Y. Lu, S. Yang, and S. Lai, "PatchNet: A simple face anti-spoofing framework via fine-grained patch recognition," in Proc. IEEE/CVF CVPR, 2022, pp. 20281–20290.

[5] Y. Sun, Y. Liu, X. Liu, Y. Li, and W. Chu, "Rethinking domain generalization for face anti-spoofing: Separability and alignment," in Proc. IEEE/CVF CVPR, 2023, pp. 24563–24574.

[6] R. Shao, X. Lan, J. Li, and P. Yuen, "Multi-adversarial discriminative deep domain generalization for face presentation attack detection," in Proc. IEEE/CVF CVPR, 2019, pp. 10023–10031.

[7] Z. Yu, J. Wan, Y. Qin, X. Li, S. Li, and G. Zhao, "NAS-FAS: Static-dynamic central difference network search for face anti-spoofing," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 9, pp. 3005–3023, 2020.

[8] Z. Zhang, J. Yan, S. Liu, Z. Lei, D. Yi, and S. Li, "A face antispoofing database with diverse attacks," in Proc. IAPR ICB, 2012, pp. 26–31.

[9] I. Chingovska, A. Anjos, and S. Marcel, "On the effectiveness of local binary patterns in face anti-spoofing," in Proc. BIOSIG, 2012, pp. 1–7.

[10] D. Wen, H. Han, and A. K. Jain, "Face spoof detection with image distortion analysis," IEEE Trans. Inf. Forensics Security, vol. 10, no. 4, pp. 746–761, 2015.

[11] Z. Boulkenafet, J. Komulainen, L. Li, X. Feng, and A. Hadid, "OULU-NPU: A mobile face presentation attack database with real-world variations," in Proc. IEEE FG, 2017, pp. 612–618.

[12] A. George, Z. Mostaani, D. Geissenbuhler, O. Nikisins, A. Anjos, and S. Marcel, "Biometric face presentation attack detection with multi-channel CNN," IEEE Trans. Inf. Forensics Security, vol. 15, pp. 42–55, 2019.

[13] M. Rostami, L. Spinoulas, M. Hussein, J. Mathai, and W. Abd-Almageed, "Detection and continual learning of novel face presentation attacks," in Proc. IEEE/CVF ICCV, 2021, pp. 14851–14860.

[14] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," Advances in Neural Inf. Process. Syst., vol. 33, pp. 6840–6851, 2020.

[15] A. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in Proc. ICML, 2021, pp. 8162–8171.

[16] I. Goodfellow et al., "Generative adversarial networks," Commun. ACM, vol. 63, no. 11, pp. 139–144, 2020.

[17] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," arXiv:1312.6114, 2013.

[18] K. He, X. Zhang, S. Ren, and S. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016, pp. 770–778.

[19] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. ICML, 2019, pp. 6105–6114.

[20] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF ICCV, 2019, pp. 1314–1324.

[21] S. Mehta and M. Rastegari, "MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer," arXiv:2110.02178, 2021.

[22] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-shot multi-level face localisation in the wild," in Proc. IEEE/CVF CVPR, 2020, pp. 5203–5212.

[23] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in Proc. IEEE/CVF CVPR, 2019, pp. 4690–4699.

[24] H. Wang et al., "CosFace: Large margin cosine loss for deep face recognition," in Proc. IEEE CVPR, 2018, pp. 5265–5274.

[25] M. Kim, A. K. Jain, and X. Liu, "AdaFace: Quality adaptive margin for face recognition," in Proc. IEEE/CVF CVPR, 2022, pp. 18750–18759.

[26] A. Mittal, A. K. Moorthy, and A. C. Bovik, "No-reference image quality assessment in the spatial domain," IEEE Trans. Image Process., vol. 21, no. 12, pp. 4695–4708, 2012.

[27] R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE ICCV, 2017, pp. 618–626.

[28] J. Maatta, A. Hadid, and M. Pietikainen, "Face spoofing detection from single images using micro-texture analysis," in Proc. IJCB, 2011, pp. 1–7.

[29] Z. Boulkenafet, J. Komulainen, and A. Hadid, "Face anti-spoofing based on color texture analysis," in Proc. IEEE ICIP, 2015, pp. 2636–2640.

[30] Y. Atoum, Y. Liu, A. Jourabloo, and X. Liu, "Face anti-spoofing using patch and depth-based CNNs," in Proc. IEEE IJCB, 2017, pp. 319–328.

[31] K. Kollreider, H. Fronthaler, and J. Bigun, "Non-intrusive liveness detection by face images," Image Vision Comput., vol. 27, no. 3, pp. 233–244, 2009.

[32] Z. Wang, Z. Yu, X. Wang, Y. Qin, J. Li, C. Zhao, X. Liu, and Z. Lei, "Consistency regularization for deep face anti-spoofing," IEEE Trans. Inf. Forensics Security, vol. 18, pp. 1127–1140, 2023.

[33] C. Liao, W. Chen, H. Liu, Y. Yeh, M. Hu, and C. Chen, "Domain invariant vision transformer learning for face anti-spoofing," in Proc. IEEE/CVF WACV, 2023, pp. 6098–6107.

[34] H. Wu, D. Zeng, Y. Hu, H. Shi, and T. Mei, "Dual spoof disentanglement generation for face anti-spoofing with depth uncertainty learning," IEEE Trans. Circuits Syst. Video Technol., vol. 32, no. 7, pp. 4626–4638, 2021.

[35] S. Liu et al., "Dual reweighting domain generalization for face presentation attack detection," arXiv:2106.16128, 2021.

[36] T. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal loss for dense object detection," in Proc. IEEE ICCV, 2017, pp. 2980–2988.

[37] X. Huang and S. Belongie, "Arbitrary style transfer in real-time with adaptive instance normalization," in Proc. IEEE ICCV, 2017, pp. 1501–1510.

[38] J. Ho and T. Salimans, "Classifier-free diffusion guidance," arXiv:2207.12598, 2022.

[39] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, "Deep learning for face anti-spoofing: A survey," IEEE Trans. Pattern Anal. Mach. Intell., vol. 45, no. 5, pp. 5609–5631, 2022.

[40] A. Vaswani et al., "Attention is all you need," in Advances in Neural Inf. Process. Syst., vol. 30, 2017.

[41] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR, 2015.

[42] C. Si, Z. Huang, Y. Jiang, and Z. Liu, "FreeU: Free lunch in diffusion U-Net," arXiv:2309.11497, 2023.

[43] Y. Liu, J. Stehouwer, and X. Liu, "On disentangling spoof trace for generic face anti-spoofing," in Proc. ECCV, 2020, pp. 406–422.

[44] A. Jourabloo, Y. Liu, and X. Liu, "Face de-spoofing: Anti-spoofing via noise modeling," in Proc. ECCV, 2018, pp. 290–306.

[45] D. Valevski, M. Kalman, Y. Matias, and Y. Leviathan, "Unitune: Text-driven image editing by fine-tuning an image generation model on a single image," arXiv:2210.09477, 2022.